

ZeroGate: Trust-Preserving Fast Paths for Governed AI Agent Runtimes

Zexun Wang*
Ond Holdings Inc.

September 2026

Abstract

AI-agent governance commonly appears as a remote approval service, a tool-permission layer, or an audit record written after execution. These controls are useful, but they force a boundary tradeoff: ordinary actions should not wait for a full governance workflow on every step, while exceptional actions must still fail closed before side effect. This paper introduces ZEROGATE Action Passes, a runtime-governance primitive inspired by the systems discipline of high-throughput contactless gates such as FeliCa/Suica. An Action Pass is a short-lived, non-reusable, locally checkable object bound to a canonical CAVA action fingerprint, granted authority, policy/state digests, holder/session identity, nonce, budget, revocation epoch, privacy posture, and minimum receipt requirement. Fresh and matching passes enter a fast lane; stale, ambiguous, verifier-dependent, colliding, privacy-sensitive, or policy/state-drifted actions enter a slow lane; expired, replayed, revoked, widened, receipt-less, holder-mismatched, or action-mismatched passes fail closed. We formalize the pass schema, gate semantics, narrowing invariant, fast-path oracle agreement, replay resistance, receipt binding, passage-envelope control, and capacity-aware routing for large action fleets. We evaluate a reference implementation through eight validation views. The synthetic oracle contains 5,760 weighted common-case observations and 360 adversarial stress cases; raw tool-permission and broad session checks false-allow 0.9000 and 0.8333 of non-fast oracle cases, while the ZEROGATE evaluator false-allows 0. A noisy 72,750-record calibration deliberately lowers aggregate fast preservation to 0.8240 while keeping modeled boundary-violation escape at 0. A public corpus of 302 GitHub Actions workflows yields 2,500 action-like records, 82.36% canonicalization coverage, and 17.64% abstain/unsupported outcomes. A passage-envelope model decomposes a 100 ms boundary budget and shows that low and moderate action-curvature profiles stay inside it only when active support prewarms state, reserves receipt/slot capacity, and coalesces slow-lane work without widening authority. A 700,000-evaluation load study over the real evaluator path reaches 64,925.28 operations per second with p95 local commit latency of 0.1838 ms and zero partial commits without receipt. The primary claim is structural: full governance is preserved as pass minting, exception routing, and proof enrichment, while the common commit path consumes a bounded action pass locally.

1 Introduction

Agentic AI systems are moving from text generation into operational consequence. They create issues, update records, send messages, call SaaS APIs, deploy code, read customer context, manipulate identity state, prepare refunds, and increasingly interact with wallets or transaction systems. The

*Correspondence: jw@nd.im

engineering question is shifting accordingly. It is no longer enough to ask whether an agent may call a tool. The stronger question is whether this exact action, under this policy snapshot, by this holder, in this runtime session, against this destination, may cross the execution boundary now.

Most current controls treat that question as a synchronous decision. The agent proposes an action, a remote service evaluates it, a human may approve it, and an audit log receives a record. This is safer than ungoverned execution, but it has two structural costs. First, it turns governance into latency and workflow friction for common actions. Second, it still leaves high-impact failures if the approval object is broad, stale, replayed, or not bound to the action that actually executes.

This paper explores a different design discipline. High-throughput gate systems such as Fel-iCa/Suica are engineered so ordinary passage can be processed locally within a short interaction window, while station and center systems still handle reconciliation, settlement, card management, and exceptional cases afterward. The card carries enough bounded state, the gate runs a short deterministic check, and later systems reconcile what happened. The important lesson is not the particular radio protocol or transit deployment. It is the system shape: prepare slow facts before the boundary; reduce the boundary check to a small local contract; keep exception handling explicit; preserve enough receipt material for later settlement and dispute.

This is also the missing link between OSuite’s earlier research primitives. CAVA gives an action a stable semantic identity [19]; PCAA defines who has authority to route, review, and prove an action [20]; BAF denotes the bounded-action frame OSuite uses for reuse, expiry, destination scope, and replay constraints; and AREG denotes the runtime exposure graph used to report controlled, degraded, observe-only, or missing runtime surfaces. ZEROGATE does not supersede any of them. It adds the commit-time object that lets their slower conclusions travel with the action.

The timing of this question is not accidental. Public standards and platform efforts are converging on agent interoperability, identity, authorization, and runtime control. NIST’s AI Agent Standards Initiative explicitly identifies interoperable, secure agents, including agent security, identity, and authorization, as a standards priority [8]. OWASP’s Agentic Applications Top 10 frames agentic systems as applications that plan, act, and make decisions across complex workflows [12]. Microsoft positions Agent 365 as an enterprise control plane for agent registry, access control, visualization, interoperability, and security [6]. MCP defines OAuth-style transport authorization, resource indicators, scopes, and step-up authorization for tool servers [7]; A2A targets interoperability between opaque agentic applications [1]; and harnesses such as the OpenAI Agents SDK expose tools, guardrails, handoffs, sessions, tracing, sandboxing, and human-in-the-loop mechanisms [10]. These layers make agents easier to connect, observe, and manage. They do not, by themselves, define a portable consumable object that says what exact action was already approved and whether that approved action is still the one crossing the side-effect boundary.

We apply that design pattern to AI-agent runtime governance. A *ZeroGate Action Pass* is a compiled authorization object for a canonical agent action. It is not a bearer credential, not a generic API token, not a model-side policy, and not an audit log. It is a bounded ticket that connects a proposed action to the authority, policy, state, scope, nonce, budget, and receipt obligations under which it may execute.

The runtime gate then has three lanes:

- **Fast lane:** the pass is valid, fresh, unused, locally sufficient, and bound to the observed CAVA action fingerprint.
- **Slow lane:** the action is not unsafe by construction, but its pass is stale, policy or state has drifted, an external verifier is required, or the pass was not compiled for local commit.

- **Fail-closed lane:** the pass is expired, replayed, budget-exhausted, invalid, widened beyond granted authority, or no longer matches the action that will execute.

This is a narrow paper. It does not propose a universal agent operating system. It does not claim that every action should be automatic. It does not replace identity, interoperability, runtime harnesses, policy engines, or audit logs. It defines the missing object between them: a locally consumable, action-bound authorization pass.

The paper makes six contributions:

1. It defines the Action Pass as a runtime-governance primitive that compiles CAVA identity, PCAA authority, BAF boundedness, and AREG exposure context into a short-lived, non-reusable, locally checkable object.
2. It formalizes fast-lane, slow-lane, and fail-closed semantics over CAVA fingerprints, authority subsets, policy digests, state digests, state predicates, revocation epochs, holder identity, nonce, aggregate budget, privacy posture, and minimum receipts.
3. It states safety properties for narrowing, replay resistance, approval-execution divergence, and receipt binding.
4. It makes fast-path decision equivalence the central invariant: the fast path is admitted only when it preserves the lane decision of full synchronous governance over the same action, policy, state, authority, and receipt conditions.
5. It positions Action Passes relative to PCAA, CAVA, BAF, AREG, A2A, MCP, IAM, OpenTelemetry-style traces, and agent harnesses.
6. It reports a reproducible non-customer evaluation suite with 5,760 weighted synthetic common-case observations, 360 adversarial stress cases, a measured local decision-function comparison, a public-runtime corpus over 302 real GitHub Actions workflow files and 2,500 action-like records, a frozen 240-record independent-labeling pack, a multi-runtime fixture corpus with 528 cases across twelve runtime surfaces, scenario baseline models, architecture ablations, tap-and-go pressure models, a passage-envelope active-support model, an action-crowd-control model, a noisy-evidence recognition calibration, an operations-research routing model, a concurrent implementation load study over the real evaluator code path, and production-path canaries that are explicitly excluded from customer SLA claims.

The evidence in this paper is intentionally stratified. The synthetic oracle and multi-runtime fixture test whether the specified semantics are implemented consistently. The public-runtime corpus measures parser coverage, abstention, and routing distribution over real workflow artifacts, but not semantic accuracy because independent labels are not yet complete. The load study measures the local evaluator path, not end-to-end hosted latency. The queueing and routing models test capacity effects under explicit assumptions rather than claiming observed customer traffic. Finally, two production-path canaries are kept separate from the benchmark tables: a hosted OSuite Studio action-recording smoke check and an externally verifiable GitHub side-effect run bound to a Baby Blue / invinoveritas signed verdict. They show that the proof path can touch live services, but they are not customer-traffic distributions or SLA measurements. This separation is important: the paper’s central claim is that a locally consumable pass can preserve a full-governance decision under stated preconditions, not that the current reference parser understands every real runtime action or that a hosted deployment already has a universal latency guarantee.

2 Design Origin: From Transit Gates to Runtime Gates

Table 1 summarizes the design translation used in this paper. The analogy is architectural rather than protocol-level: the goal is to make the side-effect boundary short, local, and explicit about exceptions while keeping settlement and proof outside the tight interaction window.

Table 1: Design translation from high-throughput gate systems to agent runtime governance.

Gate-system discipline	Runtime-governance translation	Action Pass consequence
The common gate interaction is engineered to be short.	The commit boundary cannot call every governance dependency. Exceptions and reconciliation belong outside the tight interaction window.	Verify only signed-envelope validity, pass hash, action binding, holder/session, policy/state digest, nonce, time, budget, and receipt sink.
Polling happens before passage.	The runtime can prepare identity, capability, policy, state, and receipt context before final tool arguments arrive.	The final commit checks prepared material rather than discovering all facts synchronously.
Collisions are explicit.	Concurrent use of the same resource in the same time slot is a boundary condition, not a race to ignore.	Same-slot resource conflicts leave the fast lane and route to slow review.
The user gesture is designed.	The product should make the common path feel like carrying a valid pass, not repeatedly re-litigating permission.	Operators pre-approve bounded action classes; the runtime consumes a pass at the last side-effect boundary.
Local autonomy preserves continuity.	Basic gate decisions should survive remote verifier, durable evidence, or network degradation when a fresh local pass remains sufficient.	Proof enrichment may be deferred, but receipt creation and fail-closed checks stay local.
Interrupted writes must not half-commit.	An action cannot cross if the minimum receipt sink is unavailable.	Missing receipt material blocks rather than producing an unverifiable side effect.
The card carries bounded state.	The agent carries a bounded execution ticket, not broad authority.	The pass binds holder, action, scope, policy, state, and validity.
The gate handles exceptions locally.	Drift, replay, expiry, and mismatch should not rely on model judgment.	The runtime routes to slow lane or fail-closed deterministically.
Anti-tear matters.	An action should not commit if the minimum evidence path is unavailable.	Missing synchronous receipt storage or missing persisted-before-commit evidence blocks before side effect.
Settlement is decoupled from passage.	Full proof enrichment need not block every ordinary commit.	Emit a minimal receipt synchronously; enrich proof asynchronously.
Multi-application use requires isolation.	A pass for one resource or workflow must not authorize another.	Scope widening is forbidden unless a new pass is minted.

FeliCa is widely known as Sony’s contactless IC-card technology and as the substrate behind systems such as Suica [14, 15, 17]. The two sources of inspiration are deliberately separated.

Sony’s public FeliCa material motivates the protocol-side discipline: fast polling, time-slot collision detection and avoidance, fewer transaction steps, secure local exchange within a short timing budget, multi-application access separation, and anti-tear behavior when communication is interrupted [17]. JR East’s public Suica history motivates the deployment-side discipline: railway gates needed reliable high-speed processing for roughly 24 million daily gate transactions, the fastest users kept cards in the reader field for only about 0.2 s, card-to-reader processing had to fit under roughly 0.1 s, and station gates had to continue operating autonomously when network links or servers failed [15]. Commercialization also depended on coexistence with station systems, central account and history management, stored-fare operation, and inter-enterprise settlement rather than on a purely offline gate [5, 14]. A second, weaker analogy comes from railway vehicle control: Shinkansen engineering uses suspension and body-control systems to preserve ride quality under curve and vibration conditions, rather than treating all track segments as identical straight-line motion [13, 21]. ZEROGATE borrows only this control principle. As action “curvature” rises—because of policy churn, state drift, collision pressure, revocation risk, or verifier requirements—the runtime should add active support through prewarming, reservation, coalescing, prioritization, or stopping. This paper does not depend on FeliCa internals, railway hardware, nor payment-card security properties. The transfer is architectural only: a gate can be safe and fast when slow work has already been compiled into a bounded object, the user interaction is shaped around a short local transaction, and the boundary refuses ambiguous passages immediately.

This translation matters because many agent-control systems optimize the wrong part of the problem. They make the agent easier to connect, but not necessarily easier to govern at the moment of side effect. A2A can identify and route participating agents. MCP can expose tools. OAuth can authenticate a principal. A harness can orchestrate planning, retries, memory, and tool calls. None of those layers, by themselves, make a later side effect equivalent to the action that was approved.

3 Problem Statement

Definition 1 (Action boundary). *An action boundary is the point at which an agent’s intent becomes a side effect in a system outside the model context: a file write, API mutation, SaaS state change, external message, payment, deployment, identity update, or workflow transition.*

Definition 2 (Synchronous governance tax). *A synchronous governance tax is the latency, availability dependency, and human attention cost introduced when every runtime action must synchronously consult a remote policy, approval, verifier, or audit service before it may proceed.*

Definition 3 (Compiled authority). *Compiled authority is a precomputed authorization object that captures the outcome of slower governance work and is valid only for a bounded action, holder, policy, state, scope, time, nonce, and receipt condition.*

The problem is to design a compiled-authority object for agent actions that satisfies three requirements:

1. **Local commit.** The common path must be decidable locally at the runtime boundary without invoking a model, human, remote verifier, or durable proof-export job.
2. **No silent widening.** Runtime evidence may narrow authority, route to review, or fail closed; it must not expand the authority declared in the pass.
3. **Proof compatibility.** The local decision must leave enough receipt material for later replay, proof export, and independent recomputation.

4 Critical-Path Restructuring

The performance problem is not that policy code is slow. It is that many deployments serialize unrelated governance work at the most expensive possible moment. A conventional governed agent path resembles:

$$T_{\text{sync}} = T_{\text{canonicalize}} + T_{\text{policy}} + T_{\text{risk}} + T_{\text{route}} \\ + T_{\text{approval}} + T_{\text{verifier}} + T_{\text{durable evidence}} + T_{\text{execute}}.$$

Even if each term is individually reasonable, the agent waits for all of them before the side effect can occur. This is the runtime analogue of making every passenger stop at the gate while the railway reconstructs the whole account state.

ZEROGATE changes the critical path:

$$T_{\text{commit}} = T_{\text{hash}} + T_{\text{fingerprint}} + T_{\text{freshness}} + T_{\text{authority}} + T_{\text{contention}} + T_{\text{receipt}} + T_{\text{execute/route}}.$$

The omitted work is not deleted. Identity projection, policy compilation, capability projection, risk prior, verifier route selection, and receipt-template construction move into a prepare path. Durable proof export, trace enrichment, graph updates, and external explanation move into an enrichment path. The commit path keeps only the facts that must be checked at the side-effect boundary.

This gives the paper its main engineering invariant:

$$D_{\text{fast}}(P, O) = D_{\text{oracle}}(a, \pi, x, A, E)$$

where D_{oracle} denotes the synchronous governance decision under the same policy snapshot π , state snapshot x , granted authority A , and evidence obligations E . The equality is not asserted for arbitrary systems. It holds only under the pass assumptions: the issuer envelope is valid, the pass hash is intact, the CAVA fingerprint matches, holder and runtime-session evidence match, the policy digest is fresh, the state digest and required state predicates are fresh, the revocation epoch is current, the required authority is present and a subset of the granted authority, aggregate budget evidence is sufficient, privacy minimization requirements hold, no same-slot resource conflict is present, the nonce and budget are unused, and the synchronous receipt can be persisted before side effect. Fast path is therefore not weaker governance. It is full governance compiled into local material under explicit preconditions.

The strongest latency claim is therefore topological before it is numerical. A reader can dispute the speed of a particular machine, but the commit path still has fewer online dependencies when the pass is fresh and matching.

Figures 1 and 2 make the timing claim more concrete. A railway gate feels fast because the passenger does not wait while the system reconstructs the entire fare, account, and settlement story. The system has already shaped the interaction around a bounded object, a short local transaction, explicit exception routing, and later reconciliation. ZEROGATE applies the same split to agent actions: the runtime may prepare, speculate, and enrich around the boundary, but the side-effect instant itself consumes a compact pass or refuses the action.

Table 2: Prepare, commit, and enrichment split.

Plane	Work performed	Why it is placed there
Prepare	Identity, policy digest, state digest, capability, risk prior, verifier route, receipt template.	These facts usually change slower than final tool arguments and can be prepared before commit.
Commit	Signature/envelope, pass hash, fingerprint match, holder/session, revocation epoch, digest freshness, state predicates, authority subset, aggregate budget, privacy posture, slot conflict, nonce, budget, receipt sink.	These facts decide whether intent becomes side effect now.
Enrich	Durable proof bundle, graph update, external verifier record, audit warehouse, human-readable replay.	These facts explain and prove the decision but should not block the common path once a receipt exists.

Table 3: Commit-boundary transaction step model. The number of local checks is not the win by itself; the win is that the common path has no remote dependency at the instant of side effect.

Model	Commit steps	Remote deps.	Boundary consequence
Full synchronous governance	8	4	Canonicalization, policy/state loading, verifier/approval, and durable evidence can all block passage.
ZEROGATE Action Pass	5	0	A local boundary bundle is presented, verified, consumed, receipted, then executed or routed. Fine-grained checks remain local.

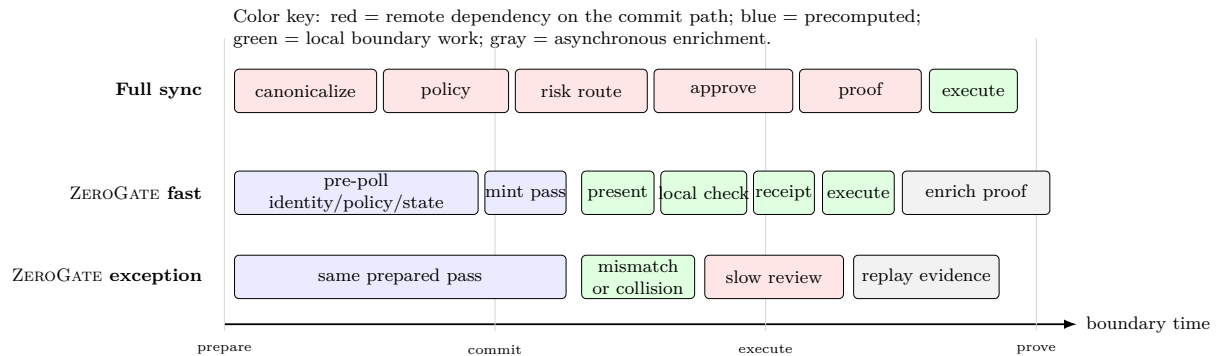


Figure 1: Boundary-time Gantt view. The intended speedup is not that ZEROGATE ignores governance. The same categories of work still exist, but most of them are moved to a prepare plane or an enrichment plane. The commit instant resembles a contactless gate: present bounded object, run a short local check, emit minimum receipt, then pass or route.

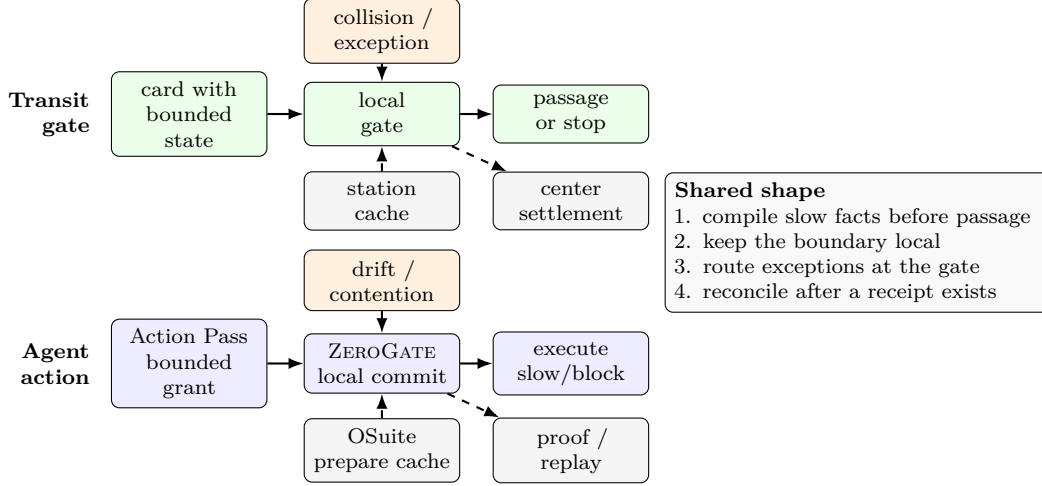


Figure 2: System-shape transfer from transit passage to agent side effects. The analogy is not that an AI action is a fare payment. The analogy is that the boundary should consume a bounded object locally while exceptional and settlement work remain outside the tight passage window.

5 Speculative Governance Without Speculative Side Effects

The prepare path can be speculative. Before the model emits final tool arguments, the runtime often already knows the task, agent identity, available tools, recent workflow state, likely destination, and policy namespace. It can therefore prefetch capability state, load policy snapshots, compute route candidates, warm verifier context, and allocate a receipt sequence. This is governance speculation, not action speculation.

The distinction is essential. ZEROGATE does not allow the runtime to execute predicted actions. It allows the runtime to precompute governance material that will be discarded unless the final action binds to the pass at the commit barrier. The final side effect still waits for the irreducible checks: action fingerprint, authority subset, policy digest, state digest, same-slot conflict state, nonce, budget, risk class, and receipt condition.

This framing gives a practical research question: what is the minimum information required at commit time to preserve a trustworthy action decision? The answer should be smaller than a full governance workflow, but larger than a tool permission bit. Action Passes are one candidate answer.

6 Threat Model and Trust Boundaries

ZEROGATE assumes that an organization has an upstream authorization layer capable of deciding whether an action may be granted. The contribution is the commit-time representation and consumption of that grant. We therefore separate the actors that can be malicious or faulty:

- **Malicious or compromised agent.** The agent may rewrite tool arguments, substitute resources, replay old approvals, omit authority evidence, or present a pass outside its intended session.
- **Compromised or weak runtime adapter.** The adapter may report incomplete runtime evidence or may only observe after execution. In that case ZeroGate can produce exposure evidence but cannot honestly claim enforce-before-execute.

- **Malicious pass presenter.** A party may copy a pass, alter fields, race two runtimes with the same nonce, or move a pass into a different workspace/session.
- **Stale policy or state store.** A pass minted under a previously valid policy or resource state may become stale before execution.
- **Receipt sink failure.** The runtime may be able to decide but unable to persist the minimum receipt before side effect.
- **Clock skew and revocation delay.** A local gate may evaluate validity intervals differently or miss an emergency revocation unless the pass is short-lived or the action is slow-laned.
- **Issuer compromise.** If the pass issuer signs overbroad or malicious passes, the local gate will faithfully consume bad authority. This is outside the gate’s protection and must be handled by issuer key management, policy review, and monitoring of minted-pass distributions.

The reference evaluator checks canonical hashes, holder/session fields, authority subsets, digests, nonces, budgets, and receipt conditions. A production deployment must add a signed issuer envelope, atomic nonce and budget consumption, adapter-mode disclosure, and durable or tamper-evident receipt storage. The paper’s benchmark evaluates the reference gate semantics; it does not claim that every possible deployment has solved issuer compromise, adapter honesty, or multi-region replay.

7 System Model

Let a raw runtime event be $a \in \mathcal{A}_r$ for runtime lane r . CAVA maps a into a canonical action object and fingerprint:

$$C(a) = (v, r, e, o, \epsilon, q, d, s, m), \quad F(a) = H(\text{JCS}(C(a))),$$

where v is schema version, e is executable or tool identity, o is normalized operation, ϵ is effect class, q is resource or subject, d is destination, s is side-effect posture, m is bounded adapter metadata, and H is a collision-resistant digest over deterministic JSON canonicalization.

Definition 4 (Action Pass). *An Action Pass is a tuple*

$$P = (id, I, Hld, S, A, F_p, D_\pi, D_x, \Phi_x, D_r, B, T, N, R, K, E, \Pi, h, \Sigma),$$

where I is issuer, Hld is holder identity, S is runtime-session binding, A is granted authority, F_p is the allowed CAVA action fingerprint, D_π is policy digest, D_x is state digest, Φ_x is the set of state predicates required for fast-lane use, D_r is revocation epoch or registry digest, B is local and aggregate consumption budget, T is validity interval, N is nonce, R is risk class, K is commit profile, E is evidence profile, Π is privacy/minimization profile, h is the canonical pass hash, and Σ is an optional signed issuer envelope. The reference implementation uses h to detect payload mutation; production authorization requires Σ or an equivalent signed envelope.

Definition 5 (Observation). *An execution observation is*

$$O = (F_o, Hld_o, S_o, A_o, D'_\pi, D'_x, \Phi'_x, D'_r, N', u, g, t, b, \sigma, \omega, \rho),$$

where F_o is the observed CAVA fingerprint, Hld_o is observed holder identity, S_o is observed runtime session, A_o is runtime-required authority, D'_π and D'_x are current policy and state digests, Φ'_x is

observed state-predicate evidence, D_r' is current revocation epoch, N' is observed nonce, u is requested unit count, g is aggregate budget evidence, t is observed time, b is runtime-provided boundary context, σ states whether the synchronous receipt sink is available, ω states whether the minimum receipt was persisted before commit, and ρ is the emitted receipt.

Definition 6 (Runtime gate). *The runtime gate is a deterministic function*

$$G(P, O) \rightarrow (\ell, y, z, \rho),$$

where $\ell \in \{\text{fast}, \text{slow}, \text{block}\}$ is lane, $y \in \{\text{execute}, \text{review}, \text{deny}\}$ is decision, z is machine-readable reason, and ρ is the minimum receipt.

Definition 7 (Authority coverage). *Granted authority is a structured tuple over operations, resource scope, destinations, systems touched, and constraints. The reference relation $A_o \sqsubseteq A_p$ holds only when every observed operation, resource, destination, and system belongs to the granted set, and every observed constraint is covered by the granted constraint. Numeric constraints use upper-bound semantics unless otherwise declared: for example, `amount_usd=73` is covered by `max_amount_usd=100`, but `amount_usd=250` is not. String and enumerated constraints require exact equality. Unknown or missing authority evidence does not satisfy the relation; it routes to slow review.*

8 Action Pass Schema

Table 4 gives the reference schema. The schema intentionally does not include natural-language rationale as an authorization field. Narrative can help an operator understand a pass; it cannot be the pass.

Table 4: Reference Action Pass fields.

Field	Example	Governance role
<code>schema_version</code>	<code>action_pass.v1</code>	Versioned pass contract.
<code>pass_id</code>	<code>ap_...</code>	Stable pass identifier.
<code>issuer.id</code>	<code>runtime-authorizer</code>	Authority that minted the pass.
<code>issuer.key_ref</code>	<code>osuite:key:prod</code>	Optional signing-key reference.
<code>holder.agent_id</code>	<code>agent:ops-bot</code>	Agent allowed to present the pass.
<code>holder.principal_id</code>	<code>operator:jw-ond</code>	Human or service principal behind the pass.
<code>holder.workspace_id</code>	<code>workspace:osuite</code>	Workspace boundary.
<code>holder.runtime_session_id</code>	<code>session:...</code>	Runtime session allowed to consume the pass.
<code>authority.requested_ref</code>	<code>skill:refund:v1</code>	Link to upstream task, skill, or policy request.
<code>authority.narrowing_rule</code>	<code>narrow-only</code>	Runtime evidence may reduce authority, not expand it.
<code>authority.granted.operations</code>	<code>issue_refund</code>	Allowed operation set.
<code>authority.granted.resource_scope</code>	<code>order:ORD-73</code>	Resource scope.

Field	Example	Governance role
<code>authority.granted.destinations</code>	<code>payment_ledger</code>	Destination scope.
<code>authority.granted.constraints</code>	<code>max_usd=100</code>	Structured limits over operation/resource/effect.
<code>action_binding.allowed_fingerprint</code>	SHA-256 hex	CAVA action fingerprint.
<code>policy_binding.policy_digest</code>	SHA-256 or digest ID	Policy snapshot compiled into the pass.
<code>state_binding.state_digest</code>	SHA-256 or digest ID	Runtime state snapshot compiled into the pass.
<code>state_binding.predicates</code>	structured object	Semantic state facts required for fast-lane use.
<code>revocation_binding.epoch</code>	<code>revocation-v7</code>	Revocation or issuer-registry freshness boundary.
<code>budget.limit / consumed</code>	<code>1/0</code>	Bounded reuse and quota.
<code>budget.aggregate_key</code>	<code>refund-day:team-a</code>	Optional shared quota or spend-bucket boundary.
<code>validity.not_before / expires_at</code>	ISO timestamp	Time-boxing.
<code>nonce</code>	random or derived nonce	Replay resistance.
<code>risk.class / score</code>	<code>low/20</code>	Fast-lane eligibility input.
<code>commit_profile.lane</code>	<code>fast</code>	Whether local commit is permitted.
<code>commit_profile.remote_verifier_required</code>	<code>false</code>	Whether a slow verifier path is mandatory.
<code>evidence_profile.synchronous_receipt</code>	<code>true</code>	Minimum receipt requirement.
<code>privacy_profile.disclosure</code>	<code>minimized</code>	Prevents raw sensitive boundary material from becoming the fast-path artifact.
<code>pass_hash</code>	SHA-256 hex	Canonical integrity hash over the pass payload.
<code>signature / envelope</code>	JWS/COSE-like	Production issuer authenticity; required for deployable authorization.

The schema is designed for composition. A skill boundary can request authority. OSuite or another runtime authorization layer can grant a narrower pass. A2A or MCP can carry the pass. A runtime harness can present the pass to a tool gateway. CAVA supplies the action fingerprint. PCAA supplies final authority and route-review-prove semantics. BAF supplies bounded reuse, expiry, and fail-closed controls. AREG explains exposure when a lane is missing, degraded, or observe-only.

9 Compatibility With CAVA, PCAA, and Skill Boundaries

Action Passes are an upgrade path, not a fork. Their position is downstream of action interpretation and upstream of side effect. This matters for compatibility.

Table 5: Compatibility map.

Layer	Existing responsibility	ZeroGate interaction
CAVA	Convert runtime-specific evidence into canonical action identity.	The pass binds to the CAVA fingerprint; if the final runtime action changes, the pass blocks.
PCAA	Assign final authority, review route, and proof obligations.	A PCAA decision can mint or deny a pass; the pass is the consumable form of that authority.
BAF	Bound reuse, scope, expiry, and execution hazards.	BAF constraints become nonce, budget, validity, destination, and fail-closed fields.
AREG	Explain runtime exposure and degraded coverage.	AREG records which runtime surfaces can fast-lane, slow-lane, block, or only observe.
Skill boundary	Vendor-neutral requested authority.	The skill requests; the runtime grants a pass that may narrow but never widen the request.

The compatibility claim is therefore strict. A skill boundary may be designed independently of OSuite. It declares requested authority. CAVA maps concrete runtime events into action identity. PCAA determines whether authority exists. ZeroGate consumes only the granted, narrowed, pass-shaped result. If the vendor-neutral skill contract improves, the pass schema can carry a better `requested_ref` and authority tuple. If CAVA improves, the pass receives a stronger fingerprint. If PCAA adds a new verifier, the pass receives a new policy/verifier digest. None of those changes invalidate the commit semantics.

This separation is also a guard against an easy design mistake. An Action Pass must not become a skill-declared permission badge. A skill can request the authority it expects; the runtime authorization layer must still decide the granted authority under task context, organizational policy, account permissions, current state, and available evidence. ZeroGate starts after that narrowing step. Its job is to consume a granted boundary without silently widening it, not to define what the boundary should have been.

Table 6: Input/output contract between the existing OSuite research primitives and ZEROGATE.

Primitive	Output consumed by ZEROGATE	Constraint on ZEROGATE
CAVA	Canonical action object and action fingerprint.	Must not reinterpret runtime semantics after pass minting; mismatch blocks.
PCAA	Granted authority, route decision, verifier requirement, proof obligation.	May consume only the granted authority; runtime observations may narrow but never widen it.
BAF	Budget, expiry, nonce, replay, and scope limits.	Must deny on replay, exhausted budget, expired validity, or widened scope.
AREG	Runtime lane coverage and exposure state.	Must disclose whether the adapter can enforce before execution or only observe afterward.
Skill boundary	Requested operation/effect/resource/destination constraints.	Treated as request input only; does not self-authorize execution.

10 ZeroGate Semantics

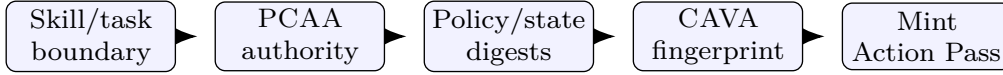
ZEROGATE is the gate semantics over the Action Pass. It is called “zero” not because no governance exists, but because the common commit path performs zero remote governance calls after the pass has been minted.

Table 7: Lane semantics.

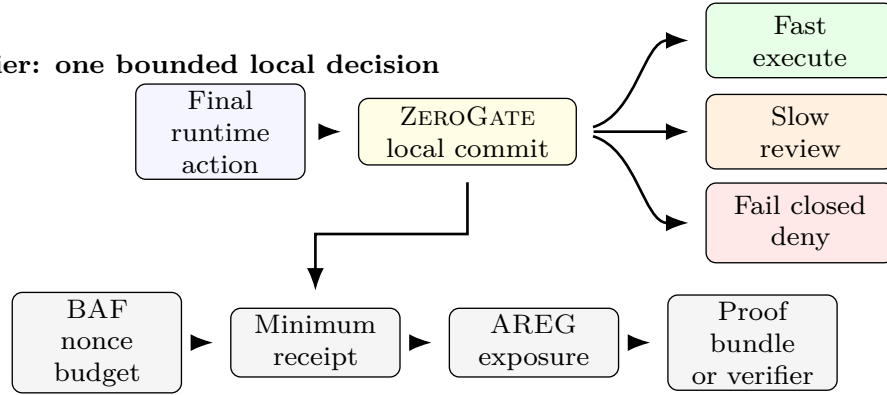
Lane	Runtime decision	Gate reason class	Required behavior
Fast	<code>execute</code>	<code>local_commit_satisfied</code>	Commit locally and emit minimum receipt.
Slow	<code>review</code>	drift or verifier dependency	Pause, re-evaluate, escalate, or ask external verifier.
Block	<code>deny</code>	invalid, replayed, expired, mismatched, widened	Fail before side effect.

Slow and block lanes may still emit a gate receipt, but that receipt records review or denial before side effect. It is not an execution receipt and does not authorize the runtime to proceed.

Prepare plane: slow facts move before the boundary



Commit barrier: one bounded local decision



Evidence plane: receipts and replay after the boundary decision

Figure 3: ZeroGate as an OSuite-compatible fast path. CAVA, PCAA, BAF, and AREG remain intact: Action Passes compile their outputs into a bounded object that can be consumed locally at the commit barrier.

Table 8: Deterministic commit-time rules in the reference evaluator.

Condition	Lane	Reason
Pass hash invalid or required fields missing	Block	invalid_action_pass
Pass not yet valid or expired	Block	pass_not_current
Pass id or action fingerprint is explicitly revoked	Block	pass_revoked
Nonce already consumed	Block	replay_nonce_seen
Observed CAVA fingerprint differs from pass fingerprint	Block	approval_execution_divergence
Requested units exceed remaining budget	Block	budget_exhausted
Requested units exceed aggregate budget evidence	Block	aggregate_budget_exhausted
Observed holder or runtime session differs from pass holder/session	Block	holder_identity_mismatch or runtime_session_mismatch
Required authority evidence is missing	Slow	authority_evidence_missing
Runtime-required authority is not covered by the pass	Block	authority_scope_widened
Synchronous receipt is required but unavailable	Block	receipt_sink_unavailable
Receipt cannot be persisted before commit	Block	receipt_not_persisted_before_commit
Current policy digest differs from pass policy digest	Slow	policy_digest_drift
Current state digest differs from pass state digest	Slow	state_digest_drift
Required state predicate is missing or drifted	Slow	state_predicate_evidence_missing or state_predicate_drift
Current revocation epoch differs from pass epoch	Slow	revocation_epoch_drift
Privacy-sensitive pass material is exposed where minimization is required	Slow	privacy_boundary_review_required
Remote verifier is required	Slow	remote_verifier_required
Pass not compiled for fast lane	Slow	not_compiled_for_fast_lane
Risk class exceeds local commit threshold	Slow	risk_exceeds_fast_lane
All local checks match	Fast	local_commit_satisfied

Table 9: Attack and drift cases covered by the gate semantics.

Case	Lane	Boundary invariant
Tool wrapper rewrite changes operation.	Block	The observed fingerprint no longer equals the approved fingerprint.
Same operation targets a different resource.	Block	Resource scope is part of the fingerprint and authority tuple.
Destination changes after approval.	Block	Destination is bound before execution, not inferred from a log.
Holder or runtime session changes.	Block	A valid pass cannot be moved to a different agent, principal, workspace, or runtime session.
Pass is presented twice.	Block	Nonce and budget make receipts historical, not reusable.
Pass was revoked after minting.	Block	Explicit revocation is stronger than ordinary freshness drift.
Revocation epoch changed.	Slow	The local gate refreshes issuer or registry state before trusting the old pass.
Aggregate spend or action budget is exhausted.	Block	A single-pass budget cannot bypass team, account, or day-level quotas.
Authority evidence is missing.	Slow	Silence is not permission; the runtime must re-evaluate rather than infer coverage.
Approval was valid but policy changed.	Slow	Policy digest drift requires re-evaluation before commit.
World state changed under the action.	Slow	State digest drift prevents stale approval from fast-laning.
Required state predicate is missing or changed.	Slow	Digest equality alone is not treated as semantic evidence when a predicate was required.
Sensitive boundary material appears in cleartext.	Slow	Fast-lane artifacts should carry minimized commitments, not raw private context.
Verifier is mandatory for this class.	Slow	External judgment remains outside the local fast path but blocks automatic commit.
Low-risk common action remains unchanged.	Fast	Local pass material is sufficient and a receipt is emitted synchronously.

11 Security Properties

Property 1 (Issuer authenticity). *In production, a fast-lane Action Pass is valid only if its signed envelope verifies under an issuer key trusted for the policy namespace and validity interval. The reference evaluator’s `pass_hash` is an integrity check over local bytes; it is not, by itself, a trusted authorization.*

Property 2 (Action binding). *An Action Pass authorizes only observations whose CAVA fingerprint equals the pass’s `action_binding.allowed_fingerprint`. If the executed action changes operation, effect, resource, destination, or material side-effect posture, the fingerprint mismatch must route to block.*

Property 3 (Runtime may narrow, never widen). *Let A_p be the authority granted by the pass and A_o be the authority required by the observed action. The fast lane is valid only if $A_o \subseteq A_p$ under operation, resource, destination, system, and constraint semantics. Missing evidence may narrow or slow the action; it may not expand the pass.*

Property 4 (Holder and session binding). *If the pass binds an agent, principal, workspace, or runtime session, the observation must present matching evidence at commit time. Omitted or mismatched holder/session evidence fails closed because a copied pass must not become portable authority.*

Property 5 (Fast-path oracle agreement). *Let D_{oracle} be the decision produced by recomputing policy, state, identity, authority, verifier route, and evidence obligations synchronously under the same inputs. Let D_{fast} be the local ZEROGATE decision. If the signed envelope is trusted, the pass hash is intact, $F_o = F_p$, holder/session evidence matches, $A_o \subseteq A_p$, $D'_\pi = D_\pi$, $D'_x = D_x$, nonce and budget are unconsumed, and a synchronous receipt can be persisted before side effect, then $D_{\text{fast}} = D_{\text{oracle}} = \text{execute}$. If any equality, subset, signature, replay, or receipt condition fails, the fast path is not available.*

Property 6 (Digest freshness). *A pass compiled under policy digest D_π and state digest D_x cannot fast-lane an observation under $D'_\pi \neq D_\pi$ or $D'_x \neq D_x$. The result is slow review rather than optimistic execution.*

Property 7 (Revocation and predicate freshness). *A pass bound to revocation epoch D_r cannot fast-lane under $D'_r \neq D_r$, and an explicitly revoked pass must fail closed. If the pass requires state predicates Φ_x , the observation must provide matching predicate evidence Φ'_x ; missing or drifted predicate evidence routes to slow review. This keeps digest freshness from becoming a blind trust shortcut when a deployment cares about specific semantic facts.*

Property 8 (Slot collision control). *If the runtime detects a same-slot conflict over a resource or sequence claim, the action must leave the fast lane even when the pass is otherwise valid. The result is slow review rather than denial because the action may still be valid after ordering, state refresh, or lease resolution. This is the agent-runtime analogue of collision detection: speed is preserved for uncontended passage, not by ignoring contention.*

Property 9 (Replay resistance). *If a nonce has already been consumed, or if the pass budget has been exhausted, the runtime must deny before side effect. A closed receipt is historical evidence, not a reusable permission object.*

Property 10 (Aggregate budget containment). *If a pass participates in a shared aggregate budget, the observed unit request must be covered by the remaining aggregate budget evidence. A locally valid*

pass cannot bypass account-level, team-level, daily, or wallet-level limits by splitting work into many individually valid passes.

Property 11 (Privacy-minimized fast lane). *If a pass declares a minimized privacy profile, the fast lane may consume hashes, predicates, labels, and policy references, but should not require raw sensitive content to be carried as pass material. If raw sensitive material is exposed at the boundary where minimization was required, the action leaves the fast lane for review or redaction.*

Property 12 (Receipt continuity). *Every gate outcome emits a minimum receipt containing pass id, pass hash, observed action fingerprint, lane, decision, reason, observed time, sequence number, and receipt hash. If the pass requires a synchronous receipt and the receipt sink is unavailable, or if the receipt cannot be persisted before commit, the action must fail closed. Later proof enrichment must bind to this receipt.*

These properties are intentionally operational rather than purely cryptographic. They define what an engineering system must preserve when connecting model output, runtime action, human authority, and later audit.

12 Operational Walkthrough

Consider an enterprise agent that is allowed to create a GitHub issue summarizing a runtime-boundary finding, but not to publish arbitrary text to other repositories, send external email, or mutate credentials. A synchronous system waits until the final tool call appears and then recomputes action identity, policy, authority, route, approval, verifier status, and evidence. ZeroGate does the opposite: it prepares what can be known early, then performs only the irreducible boundary checks at commit.

The speedup is therefore not a micro-optimization of policy evaluation. It is a change in what the runtime is asked to do at the instant of consequence. The common case does not wait for remote approval, full proof export, or human review because those concerns have already been compiled into local material. The exceptional case remains explicit because stale, widened, verifier-required, receipt-less, or mismatched actions cannot silently reuse the same pass.

Table 10: Walkthrough of one governed action.

Phase	Component	What happens
Prepare	PCAA	Resolve principal, workspace, authority route, policy version, risk posture, and verifier route.
Bind	CAVA	Canonicalize the proposed action and compute the action fingerprint.
Mint	Action Pass	Compile the allowed fingerprint, granted authority, policy digest, state digest, state predicates, revocation epoch, nonce, budget, expiry, privacy posture, and receipt obligation into a pass.
Present	Runtime	Agent presents the pass with the final action at the side-effect boundary.
Commit	ZEROGATE	Check pass hash, fingerprint, holder/session, revocation epoch, digest freshness, state predicates, authority subset, aggregate budget, privacy posture, slot conflict, nonce, budget, risk, and receipt sink locally.
Execute	Tool/SaaS	Execute only if the lane is fast; route slow cases to review and block unsafe cases before side effect.
Receipt	BAF	Emit a minimum receipt synchronously so the action is never separated from evidence.
Enrich	AREG / verifier	Add graph context, proof bundle, and external verification asynchronously without changing the commit decision.

13 Implementation

The reference implementation is implemented in `packages/cava-core/src/action-pass.js`. It exposes three public functions:

- **buildActionPass**: builds a deterministic pass from a canonical action, holder, policy, state, budget, validity, risk, and commit profile.
- **validateActionPass**: checks required fields, pass hash, and time validity.
- **evaluateActionPassCommit**: evaluates a pass against an execution observation and emits lane, decision, reason, fast-path eligibility, and receipt.

The implementation deliberately keeps the fast-lane evaluator small. It does not call a model. It does not call an external verifier. It does not query an approval database. It validates the pass hash and time window, checks holder and runtime-session binding, checks action-fingerprint equality, verifies that runtime-required authority evidence is present and covered by the granted authority tuple, compares policy and state digests, detects same-slot resource conflicts, enforces nonce and budget constraints, checks whether a synchronous receipt can be persisted before commit, and then returns a lane. This is the engineering point: expensive governance work should happen before pass minting or after receipt enrichment, not on the common gate path. The reference code intentionally does not implement a full production signing stack; the paper treats signed envelopes, key rotation, and atomic nonce stores as deployment hardening requirements around the same gate semantics.

Table 11: Primitive composition in OSuite.

Primitive	Responsibility	Action Pass dependency
CAVA	Canonical action identity and fingerprint.	Supplies the exact action hash bound into the pass.
PCAA	Deployer-owned route, review, prove loop.	Defines who may mint, approve, deny, or close the pass.
BAF	Bounded reuse, expiry, destination scope, replay control.	Supplies pass validity and fail-closed semantics.
AREG	Runtime exposure graph.	Explains which lanes are fast, slow, blocked, missing, or observe-only.
ZEROGATE	Local commit semantics.	Consumes the pass at the side-effect boundary.

14 Evaluation

14.1 Benchmark design

The benchmark is reproducible and adversarial, but it does not use private customer traffic and does not claim that all enterprise actions can fast-lane. Its purpose is to test whether the lane semantics remain stable under common, stale, widened, replayed, and evidence-degraded conditions.

The runner covers twelve action families: support-ticket note, GitHub issue creation, internal email draft, refund under limit, cloud IAM read, Web3 transfer preparation, calendar invite, CRM record update, database export preparation, Kubernetes restart, Slack announcement, and vendor-invoice scheduling. Each family is evaluated against thirty mutation classes: common case, equivalent runtime projection, durable evidence backend down, policy drift, state drift, state-predicate drift, missing state-predicate evidence, revocation-epoch drift, explicit pass revocation, tool-schema drift, same-slot resource conflict, remote verifier required, slow lane by design, high-risk scope, approval-execution divergence, resource substitution, destination substitution, authority-scope widening, expired pass, replayed nonce, budget exhaustion, aggregate-budget exhaustion, unavailable receipt sink, receipt-not-persisted, holder-identity mismatch, runtime-session mismatch, missing authority evidence, privacy-boundary exposure, not-yet-valid clock skew, and pass-hash tampering.

The benchmark reports two mixes. The *stress matrix* treats every family/mutation pair once. The *weighted synthetic common-case mix* weights common-case and equivalent-projection actions more heavily to model ordinary enterprise traffic without claiming empirical production distribution. This distinction matters: the stress matrix asks whether the gate refuses unsafe shortcuts; the weighted common-case mix asks how much synchronous governance can be removed when ordinary actions dominate. The weights are part of the artifact, not an observed customer traffic distribution.

14.2 Dataset audit protocol

To make the authored benchmark harder to overfit or misread, the artifact emits a dataset-audit block alongside the result table. The audit records a checksum of the benchmark specification, a checksum of the stress-case manifest, a checksum of the weighted representative manifest, and an oracle-leakage check over all 360 pass/observation pairs. The leakage check verifies that the evaluator input contains neither the expected lane nor benchmark mutation metadata; the lane is produced by `evaluateActionPassCommit` from pass and observation material, then compared with the held-out expected label outside the evaluator.

The artifact also reports frozen split views. Four action families—database export preparation, Kubernetes restart, Slack announcement, and vendor-invoice scheduling—are treated as held-out domain families. Eight mutation classes—state-predicate drift, revocation-epoch drift, same-slot resource conflict, authority-scope widening, aggregate-budget exhaustion, runtime-session mismatch, privacy-boundary exposure, and pass-hash tampering—are treated as held-out mutation classes. These splits do not turn synthetic data into external ground truth, but they make a narrower claim auditable: the reported exact-lane agreement does not depend only on the reference families or the easiest mutation classes.

Table 12: Frozen split audit over the synthetic stress oracle. Wilson intervals are finite-sample bounds over authored labels, not real-world semantic accuracy.

Split	Cases	Match	Fast	Zero-wait	Viol.	95% CI for match
Reference	176	1.0000	0.1364	0.6364	0	0.9786–1.0000
Held-out families	88	1.0000	0.1364	0.6364	0	0.9582–1.0000
Held-out mutations	64	1.0000	0.0000	0.5000	0	0.9434–1.0000
Held-out cross	32	1.0000	0.0000	0.5000	0	0.8928–1.0000

The same audit records zero oracle-leakage failures across all 360 stress cases. We still treat the split table conservatively: a perfect split result only means the reference evaluator matches the frozen lane oracle under these cases. It does not establish that the parser will assign the correct semantic action for every real runtime record. That question is intentionally separated into the public-runtime corpus and future human-labeled evaluation.

14.3 Public-runtime corpus methodology

To avoid making the paper depend only on author-designed cases, we add a separate public-runtime corpus. The corpus is built from real public GitHub Actions workflow files. The collection script fetches workflow YAML files from public repositories, stores the source snapshot with repository, ref, workflow path, GitHub URL, blob SHA, content hash, and raw workflow text, and then extracts action-like records from workflow steps. Each `uses:` step becomes an external-action record; each `run:` step becomes a shell-runtime record. The corpus is intentionally limited to public CI/workflow artifacts and therefore does not contain customer production traffic, private enterprise policies, or manually labeled ground truth.

The public corpus measures a different question from the synthetic oracle. It asks: when the parser sees real workflow artifacts that were not authored for the benchmark, how often can it canonicalize the action-like record, how often does it abstain, and how does the router distribute records across fast, slow, block, and unsupported lanes? It does *not* measure semantic accuracy or false-allow rate, because those require independent labels describing the intended meaning and policy posture of each workflow step. This distinction is important for IEEE-style review: the synthetic benchmark can have exact oracle agreement because the oracle is defined by construction; the public-runtime corpus should not be expected to produce all-1.0000 safety metrics. The artifact therefore stores both the source-workflow snapshot checksum and a derived-record checksum, reports Wilson intervals for coverage proportions, uses a deterministic repository-level reference/held-out split, and displays a stratified sample across repositories, lanes, and action families rather than the first records in file order.

The same corpus generator emits a 240-record labeling pack. The labels are intentionally blank in the release used by this paper. The pack is designed for an independent annotation round in

which annotators label intended action family, operation, effect, resource, destination, evidence completeness, expected boundary outcome, confidence, and notes. Only after those labels are frozen should the artifact report runtime-to-action mapping accuracy and authorization-decision accuracy over the public corpus. This is a small but important reproducibility choice: it prevents the paper from turning coverage into an accuracy claim before independent labels exist.

Table 13 states the intended annotation protocol so that a future labeled result can be audited rather than retrofitted. The protocol separates the two tasks that matter for this paper. The first task asks whether the runtime record was mapped to the intended canonical action. The second asks whether the authorization lane is correct given a frozen policy interpretation. A system can be good at one and weak at the other; combining them into one number would hide the failure mode.

Table 13: Independent-labeling protocol for the public-runtime corpus. The current paper releases the pack and protocol; it does not report the labeled accuracy until independent labels are complete.

Item	Protocol
Sampling unit	240 frozen records stratified by repository, runtime record type, parser lane, parser status, and action family.
Annotators	At least two annotators who are not the parser author and do not modify the evaluator during labeling; a third adjudicator resolves disagreements.
Blind material	Annotators see source workflow context and raw step text, but not the parser’s internal reasoning, expected benchmark labels, or post-hoc implementation changes.
Labels	Intended operation, effect, resource, destination, evidence completeness, expected boundary outcome, confidence, and free-text rationale.
Agreement	Report Cohen’s κ for two annotators or Krippendorff’s α for three or more before reporting accuracy.
Freeze rule	Labels, adjudication rules, and held-out split are frozen before parser or policy-evaluator changes are made against the pack.
Reported metrics	Runtime-to-action mapping accuracy, authorization-decision accuracy conditioned on frozen labels, false-allow rate, false-block rate, correct review/escalation rate, and cross-runtime consistency where applicable.

14.4 Multi-runtime fixture methodology

The public corpus is useful because it is non-synthetic, but it is limited to GitHub Actions. We therefore add a separate multi-runtime fixture corpus to test a narrower interface claim: if a vendor-neutral skill boundary is projected into different runtime shapes, can the same Action Pass semantics be consumed consistently? The fixture includes twelve runtime surfaces—MCP tool call, SDK call, shell command, workflow node, browser DOM action, SaaS API, Web3 transaction, A2A message, agent harness hook, Dify workflow, n8n workflow, and DeepSeek Harness plugin—four skill families, and eleven variants per family/runtime. The variants cover equivalent projection, runtime narrowing, policy drift, missing state predicates, privacy exposure, resource substitution, destination widening, operation widening, holder/session mismatch, receipt-sink failure, and ambiguous runtime evidence.

This fixture is not a replacement for empirical runtime corpora. Its value is that it separates two measurements that are often conflated: whether the runtime event is mapped to the intended CAVA action, and whether the authorization decision is correct after mapping. Perfect results on this fixture mean only that the boundary semantics are stable under these frozen projections. They do not establish that every real MCP, browser, workflow, or harness record will be parsed correctly.

14.5 Metrics

We report five governance-specific metrics:

$$\text{FPC} = P(\ell = \text{fast}), \quad \text{ZWR} = P(\ell \in \{\text{fast}, \text{block}\}),$$

$$\text{SFPC} = P(\ell = \text{fast} \wedge \ell = \ell_{\text{oracle}}), \quad \text{VR} = P(\ell \neq \ell_{\text{oracle}}),$$

where ℓ_{oracle} is the lane expected from full synchronous governance over the same action, policy, state, authority, holder/session, and receipt conditions. We also report receipt-object construction, persisted-before-commit rate, fast-commit receipt integrity, and partial commits without receipt. These are separate on purpose: a deny or review decision can still construct a gate receipt, while an executed action must have persisted receipt material before the side effect. FPC measures how often the system can proceed without slow governance. ZWR measures how often the runtime reaches a terminal decision without waiting: fast execution or immediate denial. SFPC is the central safety metric; it prevents a system from improving FPC by admitting unsafe fast paths.

14.6 Timing and load-study methodology

The timing result is intentionally secondary. The main evaluation asks whether the lane decision is preserved while remote dependencies are removed from the common commit boundary. To avoid presenting a fragile single-call stopwatch result as a system guarantee, the artifact also runs a batch timing study: it preconstructs the 360 stress-matrix pass/observation pairs, warms the evaluator for 8 rounds, then measures 40 full rounds using `process.hrtime.bigint`. Each round evaluates all 360 cases and records the mean milliseconds per operation.

This study measures only the local commit evaluator on the local runner (Node v22.18.0, macOS arm64, Apple M4 Pro). It excludes pass minting, adapter parsing, durable storage, network latency, verifier calls, UI approval, tool execution, queueing, and production contention. The artifact records `production_sla_claim=false`. The timing number is therefore a sanity check that the local gate is small; the stronger and portable claim is the transaction-step model in Table 3.

We also run a concurrent implementation load study. Unlike the tap-and-go pressure model, this study executes the published `packages/cava-core` implementation path: workers import `buildActionPass` and `evaluateActionPassCommit`, construct bounded passes, and evaluate deterministic runtime observations covering common passage, policy drift, revocation drift, explicit pass revocation, state-predicate drift, same-slot conflict, action mismatch, replay, aggregate-budget exhaustion, missing receipt sink, receipt persistence failure, holder/session mismatch, missing authority evidence, privacy-boundary exposure, and authority widening. The workload inputs are still synthetic for reproducibility; the evaluator, receipts, lane decisions, and failure reasons are not mocked. This distinction is important: the study is not live customer production traffic, but it is a real code-path stress run rather than a hard-coded table.

Finally, we keep a live Studio smoke check separate from both benchmark tables. In that check, the OSuite CLI configuration points to `https://studio.osuite.ai`; `osuite status` returns HTTP 200; and a low-risk research action record is created and read back through the hosted `/api/actions` path. This validates hosted action-recording reachability, but it is excluded from all fast-lane performance claims because it measures a different production surface.

14.7 Production-path evidence and SLA boundary

Three forms of “real” evidence are easy to confuse. The first is real public runtime data, such as the GitHub Actions corpus. That evidence is useful for coverage and abstention, but it is not customer production traffic and has no independent semantic labels yet. The second is a production-path canary: a bounded action crosses a live service path and leaves recomputable evidence. The third is a customer SLA: repeated end-to-end measurements over a deployed tenant under defined load, region, adapter, storage, and tool-execution conditions. This paper includes the first two categories and explicitly does not claim the third.

Table 14: Production-path evidence ladder. The table distinguishes live-service reachability and public side-effect binding from customer SLA claims.

Evidence	Observed path	Supports	Does not support
Public workflow corpus	302 real public GitHub Actions workflow files yielding 2,500 action-like records.	Parser coverage, abstention, unsupported surfaces, and routing distribution over public artifacts.	Customer traffic distribution, semantic accuracy, or false-allow rate without labels.
Hosted Studio smoke	<code>osuite status</code> returned HTTP 200; <code>POST /api/actions</code> returned 201; <code>GET /api/actions/{id}</code> returned 200 for action <code>act_86a9196a...</code>	Live hosted action-recording reachability and read-back persistence for a low-risk research action.	Fast-lane latency, external SaaS execution, customer load, or enforcement inside a third-party runtime.
External verifier and SaaS side effect	OSuite created a CAVA artifact for a public GitHub issue, received a Baby Blue / <code>invinoveritas v11 approve_with_concerns</code> verdict, self-submitted signed ledger entry 246, and created GitHub issue 2; the issue body hash matched the approved body hash.	A real third-party SaaS side effect can be bound to a pre-action artifact, signed external verdict, public ledger entry, and post-execution content hash.	Who physically clicked or called GitHub, customer SLA, identity attestation beyond the stated account evidence, or universal semantic parser accuracy.
Tenant SLA run	Not claimed in this paper.	Future measurement should report end-to-end distributions by adapter, region, storage mode, verifier mode, and tool class.	Not available from the current reproducible artifact alone.

The end-to-end SLA question is therefore not whether the local evaluator is fast; it is which clocks are included. A production SLA must measure at least five intervals: pass preparation, boundary commit, receipt persistence, tool or SaaS execution, and proof enrichment. For the common lane, the proposed SLA statistic should report p50/p95/p99 for commit-plus-receipt under warm and cold adapter state. For the slow lane, it should separately report queue wait and reviewer/verifier time. For block, it should report time-to-denial and whether a denial receipt was persisted. Mixing these clocks into one average would make the result easier to market but less useful for operators.

This structure is deliberately conservative. It lets the paper claim that the common boundary decision can be made locally, that the real code path has low local overhead, and that live canaries can bind to real service paths. It prevents the stronger but unsupported claim that every hosted deployment already meets a fixed end-to-end SLA under customer load.

14.8 Results

The exact-lane metrics in Table 16 are regression-oracle metrics over a synthetic benchmark. They are useful for checking whether the implementation preserves the specified lane semantics, but they should not be read as real-world semantic accuracy. Table 17 reports the separate public-runtime

Table 15: End-to-end SLA protocol required before claiming hosted production latency.

SLA component	Measurement	Reporting rule
Prepare	Policy, state, capability, verifier-route, and receipt-template preparation before the final side-effect boundary.	Report separately from commit so prefetch quality is visible.
Commit	Signature/envelope, pass hash, action fingerprint, holder/session, authority, digest, nonce, budget, privacy, slot, and receipt-sink checks.	The only component compared with the local evaluator benchmark.
Receipt persistence	Minimum receipt write before side effect.	Fast execution is invalid if the minimum receipt is missing or not persisted before commit.
Tool/SaaS execution	External API, shell, workflow, wallet, or browser side effect.	Report by adapter class; do not include in local gate latency.
Proof enrichment	Replay bundle, external verifier, graph update, and human-readable explanation.	May be asynchronous; report completion lag separately from execution permission.

Table 16: Synthetic regression-oracle benchmark summary. These are lane-equivalence checks over authored stress cases, not real-world semantic accuracy.

Metric	Value
Weighted synthetic common-case observations	5,760
Stress matrix cases	360
Action families	12
Mutation classes	30
Weighted common-case exact lane match	1.0000
Weighted common-case fast-path coverage	0.7500
Safety-preserving fast-path coverage	0.7500
Zero-wait ratio	0.8771
Violation rate	0.0000
Receipt object constructed rate	1.0000
Receipt persisted-before-commit rate	0.9917
Fast commit receipt integrity	1.0000
Partial commits without receipt	0.0000
Stress exact lane match	1.0000
Stress fast-path coverage	0.1000
Batch timing operations	14,400
Batch timing mean ms/op	0.0822
Batch timing 95% CI ms/op	[0.0798, 0.0846]
Concurrent load-study operations	700,000
Concurrent load-study throughput ops/s	64,925.28
Concurrent load-study p95 ms/op	0.1838

corpus, where the parser encounters real workflow artifacts and must abstain when evidence is too complex or unsupported.

Table 17: Public-runtime corpus over real public GitHub Actions workflow files. These are coverage and routing-distribution metrics, not accuracy metrics.

Metric	Value
Public repositories with fetched workflow files	28
Workflow files	302
Action-like records	2,500
Canonicalized records	2,059
Abstain or unsupported records	441
Canonicalization success rate	0.8236
Abstain or unsupported rate	0.1764
Fast lane rate	0.3848
Slow lane rate	0.4372
Block lane rate	0.0016
Unsupported lane rate	0.1764
Mutable external action rate	0.0976
High-risk shell rate	0.0336
Complex shell abstention rate	0.1196
Canonicalization success 95% CI	[0.8082, 0.8380]
Abstain/unsupported 95% CI	[0.1620, 0.1918]
Fast-lane 95% CI	[0.3659, 0.4040]
Repository held-out canonicalization	0.8629
Repository held-out records	715
Measured semantic accuracy	not measured
Measured false-allow rate	not measured
Labeling-pack records	240

The public corpus changes how to read the paper’s claims. On real public workflow records, the reference parser does not claim complete coverage: 17.64% of action-like records are abstained or unsupported, and 43.72% route to slow lane rather than fast execution. This is the desired posture for an early reference parser. It demonstrates that ZeroGate can preserve a fast lane without pretending that every real runtime artifact is locally understood.

Table 18: Multi-runtime fixture corpus. This is a regression fixture for boundary preservation across runtime shapes, not an empirical parser-accuracy corpus.

Metric	Value
Runtime surfaces	12
Skill families	4
Variants per family/runtime	11
Fixture cases	528
Fast / slow / block cases	96 / 144 / 288
Runtime-to-action mapping accuracy	1.0000
Authorization-decision accuracy	1.0000
Case manifest sha256	3cd1ce4d...393330

The multi-runtime fixture is deliberately placed next to, but not merged with, the public corpus. The public corpus answers a coverage question over real GitHub Actions artifacts. The fixture answers a semantic-interface question over MCP, SDK, shell, workflow, browser, SaaS API, Web3, A2A, harness, Dify, n8n, and DeepSeek Harness-shaped events. A reviewer should not average these tables into one accuracy number.

The modeled false-allow column should be read as a scenario-oracle quantity: it counts cases in this benchmark that would be unsafe if a baseline admitted any available tool call or broad session approval. It is not a measured defect rate for any named vendor product.

Table 19: Lane counts.

Lane	Weighted common-case mix	Stress matrix
Fast	4,320	36
Slow	708	144
Block	732	180

Table 20: Scenario baseline model. Non-ZEROGATE rows are explicit modeled baselines, not measured competing systems. Latencies for remote full governance are scenario assumptions; ZEROGATE latency is the diagnostic local commit p50 and is not a production SLA.

System	Latency ms	ZWR	Modeled false allow	Decision eq.
Raw tool permission	0.020	1.0000	0.2500	0.7500
Audit only	0.040	1.0000	0.2500	0.7500
Session approval	0.080	1.0000	0.1537	0.8463
Full synchronous governance	420.000	0.0000	0.0000	1.0000
ZEROGATE Action Pass	0.0840	0.8771	0.0000	1.0000

Table 21: Measured local decision-function comparison over the same 360 pass/observation pairs. These are local decision functions, not closed-vendor product benchmarks.

System	p50 ms	p95 ms	False allow	False block	Evidence	Decisions
Raw tool permission	0.000055	0.000202	0.9000	0	0	360 exec
Broad session approval	0.000072	0.000143	0.8333	0	0	336 exec / 24 deny
ZEROGATE evaluator	0.086190	0.133322	0	0	1	36 exec / 144 review / 180 deny

The table separates two different meanings of “fast.” IAM or tool-scope checks can be faster than ZEROGATE because they answer a smaller question: whether a caller has capability. ZEROGATE intentionally spends more local work than a raw capability gate because it also checks action identity, holder/session binding, digest freshness, authority coverage, replay, budget, privacy posture, and receipt conditions. The meaningful comparison is therefore not “faster than a boolean permission bit.” It is faster than forcing every consequential action through full synchronous governance while preserving the same lane oracle for the cases admitted to the fast path.

Table 22: Mainstream governance-pattern comparison. Non-ZEROGATE rows are scenario models for common industry designs, not measurements of any named closed-source vendor system.

Pattern	Remote deps.	p50 ms	ZWR	Action binding	Replay receipt	Primary gap
OSuite v0 synchronous review	4	420.000	0.0000	yes	yes	Strong semantics, but every action pays full remote governance latency.
Vendor-native control plane	2	160.000	0.4500	platform-scoped	export-dependent	Strong inside one trust boundary; weaker as an independent cross-runtime proof object.
Harness policy hook	1	35.000	0.7000	payload-dependent	no	Useful orchestration hook, but retries and rewrites can move away from the approved object.
IAM or MCP scope gate	0	0.020	1.0000	no	no	Answers capability and scope, not exact action ownership.
ZEROGATE Action Pass	0	0.084	0.8771	yes	yes	Requires correct pass minting, CAVA mapping, fresh digests, and receipt storage.

Table 23: Sensitivity analysis for the scenario-model full-governance latency. The local p50 comes from the diagnostic single-case evaluator timing; the synchronous p50 column is intentionally varied rather than asserted as a measured production value.

Full sync p50 ms	Local p50 ms	Reduction	Remote deps. removed
50	0.0840	0.9983	4
100	0.0840	0.9992	4
250	0.0840	0.9997	4
420	0.0840	0.9998	4
1000	0.0840	0.9999	4

Table 24: Concurrent implementation load study. This run exercises the actual Action Pass evaluator path under worker concurrency; it is not live customer traffic.

Metric	Value
Workers	14
Operations per worker	50,000
Total boundary evaluations	700,000
Throughput ops/s	64,925.28
Fast / slow / block counts	483,183 / 114,605 / 102,212
Expected lane match	1.0000
False allow rate	0.0000
Receipt object constructed rate	1.0000
Receipt persisted-before-commit rate	0.9867
Partial commit without receipt	0.0000
Duplicate nonce block rate	1.0000
Same-slot conflict slow-review rate	1.0000
Missing receipt sink block rate	1.0000
Receipt-not-persisted block rate	1.0000
Missing authority evidence slow-review rate	1.0000
Holder/session mismatch block rate	1.0000 / 1.0000
Revocation block / epoch slow-review rate	1.0000 / 1.0000
State predicate slow-review rate	1.0000
Aggregate budget block rate	1.0000
Privacy boundary slow-review rate	1.0000
Commit latency p50 / p95 / p99 ms	0.0803 / 0.1838 / 0.4751

Table 25: Architecture ablations.

Variant	FPC	False allow	Main loss
Full ZEROGATE	0.7500	0.0000	None in this oracle.
Without action binding	0.9610	0.0505	Operation, resource, and destination drift can pass.
Without policy digest	0.8440	0.0183	Policy drift becomes invisible.
Without state digest	0.8440	0.0183	Stale state can reuse approval.
Without nonce/budget	0.8490	0.0183	Replay and repeated spend become possible.
Without holder/session binding	0.7667	0.0167	Copied passes become transferable authority.
Without revocation binding	0.7604	0.0104	Emergency revocation and registry freshness gaps become invisible.
Without state predicates	0.7625	0.0125	Semantic state requirements can be missing or stale.
Without receipt persistence gate	0.7667	0.0167	Side effects can execute without durable minimum evidence.
Without fast/slow split	0.0000	0.0000	Safe but every action pays full synchronous tax.
Without async evidence	0.7500	0.0000	Decision remains safe, but proof construction returns to the critical path.

14.9 Tap-and-go pressure model

The benchmark also includes a deterministic tap-and-go simulation. This is not a production throughput claim. It is a pressure model for the user experience that ZEROGATE is trying to create: ordinary bounded actions should feel like passing a local gate, while collisions, stale state, replay, missing receipts, and high-risk cases should visibly leave that gate.

Table 26: Tap-and-go simulation. ZEROGATE p95 is an expected boundary wait under the weighted common-case lane mix plus scenario-specific churn and conflict; synchronous p95 models a remote full-governance check on every action.

Profile	Actions	Fast	Slow	Block	Tap-go	ZG p95 / Sync p95
Single agent repeated tap	120	90	14	16	0.8833	115.0 / 980
Team burst	480	343	71	66	0.8521	145.6 / 980
Enterprise gate hour	9,600	6,900	1,419	1,281	0.8522	145.5 / 980
Incident policy churn	1,200	630	397	173	0.6692	324.7 / 980

The table shows why the design is experienced as fast even when not every action fast-lanes. In the single-agent and normal-enterprise profiles, about 85–88% of actions reach a terminal boundary decision without waiting for remote governance. The incident profile deliberately increases policy churn and contention; fast-path coverage drops, but the system still does not become a raw allow/block filter. It slows the right actions down instead of pretending the pass remains clean.

Collision and anti-tear checks are evaluated separately across all twelve action families. Duplicate nonces and explicit revocations are blocked in all cases; same-slot resource conflicts, revocation-epoch drift, state-predicate drift, missing state-predicate evidence, privacy-boundary exposure, and missing authority evidence route to slow review in all cases; missing synchronous receipt sinks and missing persisted-before-commit evidence are blocked in all cases; holder/session mismatches are blocked in all cases; aggregate-budget exhaustion is blocked in all cases; and no simulated action commits without a receipt. This is the FeliCa-inspired part that matters for agents: the fast gate is fast

because it has a short local transaction, not because it ignores collision, revocation, quota, privacy, or partial-write risk.

The long-workflow model makes the same point at workflow scale. For 64, 256, and 1,024-step workflows, the expected governance wait falls by roughly 87–88% compared with forcing every step through full synchronous governance, while preserving the same slow/block exceptions. The exact numbers are model outputs under stated assumptions, but the qualitative lesson is robust: synchronous governance latency compounds with agentic workflow length; local consumable passes stop that compounding for the common path.

The stress matrix is intentionally adversarial: most cases are drift, replay, expiry, verifier, widened-authority, receipt, or divergence cases. The low stress fast-path coverage is therefore a feature, not a failure. It shows that the reference evaluator does not aggressively force ambiguous actions into the fast lane. The weighted common-case mix tests a different question: if ordinary low-ambiguity actions dominate, can the system preserve local commit for those actions while still routing exceptions away from fast execution? In this benchmark, it can.

14.10 Passage envelope and active support

The tap-and-go model above is still too coarse to explain why the boundary feels fast. Railway-ticketing systems do not merely say that a gate should be “low latency.” They engineer a short interaction envelope, reduce transaction steps inside it, and move preparation or settlement work outside it. We translate that discipline into a *passage envelope*: a 100 ms common-boundary budget for the local decision that determines whether a proposed action executes, slows, or stops. This is a design envelope, not a claim about hosted OSuite SLA or JR East hardware equivalence.

Table 27 decomposes the envelope. The measured p95 local evaluator time in the batch timing run is 0.0969 ms, using 0.0969% of the 100 ms envelope. This is intentionally reported as a local function measurement. Adapter parsing, pass minting, storage, network calls, external verifiers, UI approval, and tool execution are outside the number. The point is not that every end-to-end action completes in 0.0969 ms; it is that the commit decision can be made small enough to be treated like a gate transaction once the slow facts have already been compiled into the pass.

Table 27: Passage-envelope decomposition. Budgets sum to a 100 ms common-boundary envelope. Estimated p95 stage costs are proportional decompositions of the measured local evaluator p95, not independently instrumented per-stage timers.

Stage	Budget ms	Est. p95 ms	Active support before commit
Present pass and observation	8	0.0078	Pre-assemble holder/session and final action observation.
Verify envelope and hash	18	0.0175	Keep issuer envelope and pass hash in local material.
Match fingerprint and authority	24	0.0271	Compile CAVA fingerprint and authority projection before side effect.
Check freshness and slot	22	0.0233	Cache policy/state digests, revocation epoch, and per-resource slot state with freshness limits.
Consume nonce, budget, receipt	18	0.0155	Commit nonce, aggregate budget, and minimum receipt before side effect.
Dispatch execute or route	10	0.0058	Return a terminal local lane: execute, slow-review, unsupported, or block.

The Shinkansen-inspired part is the control law rather than the hardware. A train entering a curve needs more support than one moving along a straight segment; the support must arrive before the passenger feels the curve. In ZEROGATE, the analogous quantity is *action curvature*: policy churn, state drift, same-slot collision pressure, revocation risk, evidence ambiguity, or verifier

requirement. Active support may prewarm digests, reserve receipt capacity, reserve resource slots, coalesce related conflicts, move reviewer capacity forward, or stop stale passes. It may not widen authority. If support would make an unauthorized action appear fast, it is not support; it is a bypass.

Table 28: Curvature-aware passage-envelope model. “Before support” is a static fast gate with FIFO slow lane; “after support” adds prewarming, slot reservation, coalescing, and capacity reservation without widening authority.

Profile	Actions	Curv.	P95 before	P95 after	Envelope	Meaning
Straight-line common ops	9,600	0.07	59.8	18.1	yes	Prepared policy/state and receipt capacity keep common actions local.
Baggage and dependent work	14,400	0.14	115.7	32.2	yes	Context is folded into receipts; dependent work inherits narrowed authority.
Rush-hour resource collision	18,000	0.31	223.5	68.4	yes	Slot reservation and coalescing keep contention visible without reviewing every item.
Sharp policy-churn curve	6,000	0.58	384.2	206.0	no	A controlled slowdown is correct when policy/state changes quickly.
Emergency revocation curve	2,400	0.86	205.8	142.2	no	Revoked or stale passes should stop even if the envelope is missed.

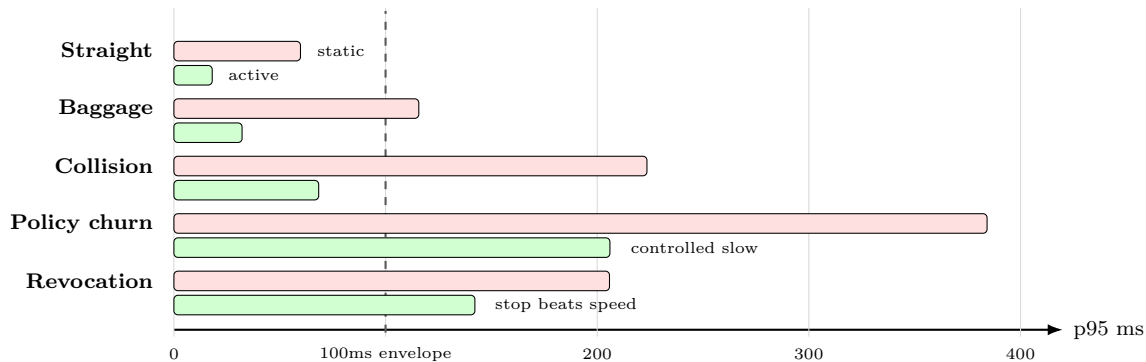


Figure 4: Active-support curve. Low and moderate action curvature can stay under the common passage envelope after prewarming, slot reservation, and coalescing. High policy churn and emergency revocation intentionally exceed the envelope because safety requires slow review or stop.

The practical consequence is that ZEROGATE is not just a faster if-statement. It is a boundary controller. For straight-line traffic, it removes avoidable remote dependencies. For baggage-like evidence and dependent subactions, it avoids turning every supporting item into a separate review. For collision pressure, it reserves and coalesces slots before reviewers are overwhelmed. For policy churn or emergency revocation, it deliberately breaks the tap-and-go feel. This is the intended invariant: comfort is optimized only inside the safety envelope.

14.11 Action crowd control

High-throughput gates are not only fast because they process a token quickly. They must also classify boundary context. A suitcase, stroller, or companion moving with a passenger is not the same object as a second fare-bearing passenger; a collision or ambiguous passage should not be silently counted as a normal entry. The public Suica/FeliCa material used in this paper supports

Table 29: Passage-envelope comparison across design patterns. Non-ZEROGATE rows are explicit scenario models; ZEROGATE local timing is measured only for the reference evaluator.

System	Mean p95 ms	Profiles inside envelope	False allow	Boundary weakness
Linear full governance	980.0	0 / 5	0	Safe, but ordinary actions miss the passage envelope.
Static fast gate, no support	197.8	1 / 5	0	Safe, but burst and collision pressure leak into user-facing wait.
ZEROGATE active support	93.4	3 / 5	0	Common curves fit; emergency and high-assurance windows still slow or stop.
Raw permission gate	0.05	5 / 5	0.25	Fast because it does not bind exact action, freshness, receipt, or divergence.

the broader system constraint—short interaction windows, high daily gate volume, explicit collision handling, autonomous local operation, and later settlement—but we do not rely on a public claim about JR East’s internal passenger-classification implementation. Instead, we translate the design lesson conservatively: a governed agent runtime must distinguish action-bearing side effects from context, dependent work, piggybacked side effects, and resource collisions.

This matters once an organization operates hundreds or thousands of agents. A naive system that linearly reviews every action-like item creates a governance queue even when many items are context baggage or dependent subactions. A naive tool-permission system has the opposite failure: it lets anything shaped like an allowed tool call through, including piggybacked side effects and same-slot conflicts. ZEROGATE aims for the middle: classify the stream before review, fold context into the primary receipt, contain dependent work under the narrowed pass, block tailgating actions, and route collisions or ambiguous boundaries to slow review.

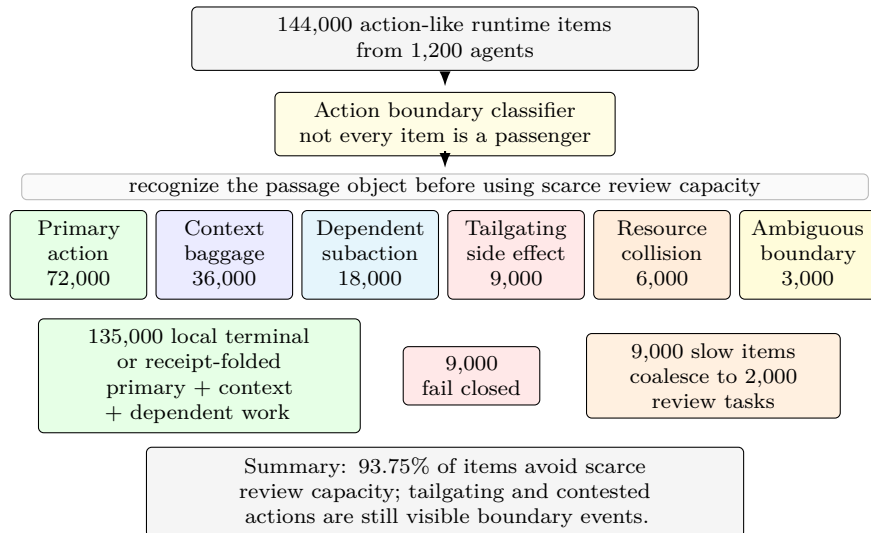


Figure 5: Action-stream recognition before review. The model mirrors the gate-design lesson that throughput requires recognizing the passage object, not simply counting every moving item. ZEROGATE separates primary side effects from context baggage, dependent work, piggybacked side effects, resource collisions, and ambiguous evidence before scarce human or verifier capacity is used.

In this model, class-level recognition accuracy is 1.0000 against the specified oracle; false-passenger

Table 30: Action crowd-control model over 144,000 action-like items from 1,200 agents. The model is deterministic and reproducible; it is a recognition stress model, not observed customer production traffic.

Class	Gate analogy	Count	Handling	Interpretation
Primary action	Passenger	72,000	Pass-bound fast or policy-slow	The side effect that must carry authority.
Context baggage	Luggage	36,000	Attach to receipt	Evidence travels with the action but is not an independent side effect.
Dependent subaction	Child or companion	18,000	Contain under pass	Helper work inherits narrowed authority and cannot widen scope.
Tailgating action	Fare evasion	9,000	Block	A second side effect tries to ride through on the wrong pass.
Crowd collision	Same-slot conflict	6,000	Slow review	Multiple agents contend for the same resource, quota, wallet, or state transition.
Ambiguous boundary	Unclear passage	3,000	Slow review	Evidence cannot prove whether the item is context, dependency, or new side effect.

Table 31: Nonlinear action-stream handling. A serious runtime cannot scale by reviewing every item, but it also cannot collapse every item into a permitted tool call.

System	Reviews	Blocks	Local terminal	Failure or advantage
Linear per-action review	144,000	0	0	Treats context and dependent work as separate passengers, creating avoidable queues.
Tool-permission counter	0	0	144,000	Treats every permitted tool-shaped item as safe, including tailgating and collisions.
ZEROGATE crowd control	9,000	9,000	135,000	Folds baggage into receipts, contains dependencies, blocks piggybacked actions, and slows contested slots.

rate is 0; tailgater block rate is 1.0000; baggage-context preservation is 1.0000; dependent-action containment is 1.0000; and collision slow-review rate is 1.0000. These numbers are not a claim about universal semantic understanding. They show that the reference action-boundary model can represent the distinction that throughput alone would miss.

14.12 Noisy evidence calibration

To make the recognition claim less brittle, the artifact also includes a noisy-evidence calibration. It keeps the same class taxonomy but deliberately degrades the evidence available at the boundary: wrappers rename operations, state evidence disappears, free-text labels include Japanese or other locale-specific terms, many agents contend for the same resource slot, and near-miss actions preserve surface shape while changing amount, destination, identity, or effect. This is still a deterministic calibration, not a customer-traffic measurement and not semantic accuracy on independently labeled traces. Its purpose is to measure the safety behavior readers should expect when the gate is uncertain.

Table 32: Noisy recognition calibration. Non-perfect fast preservation is expected; the safety target is that uncertainty leaves the fast lane instead of becoming a boundary-violation escape.

Profile	Records	Match	Fast kept	Review/abstain	Unsupported	Escape	Main effect
Clean structured runtime	12,000	0.9710	0.9640	0.0915	0	0	Most typed evidence remains local.
Wrapper obfuscation	12,000	0.8662	0.8500	0.1963	0	0	Renamed tools reduce fast-lane preservation.
Missing state evidence	12,000	0.7333	0.7000	0.3292	0	0	Freshness gaps create a deliberate uncertainty tax.
Locale/free-text noise	12,000	0.8225	0.8000	0.2400	525	0	Structured fields matter more than language labels.
Same-slot crowd pressure	12,750	0.9169	0.9000	0.2008	0	0	Contention increases slow-lane load.
Mixed adversarial near miss	12,000	0.7600	0.7300	0.3025	0	0	Near-miss drift is costly but does not fast-lane.
Aggregate	72,750	0.8457	0.8240	0.2264	525	0	Fast path degrades conservatively under noise.

The important number in Table 32 is not only the 0.8240 aggregate fast-preservation rate. It is the combination of reduced fast preservation, increased review/abstain, and zero boundary-violation escape under the modeled profiles. A system that maintains a high fast-path number by admitting incomplete evidence is not behaving like a reliable gate; it is behaving like a permissive counter. A system that routes every uncertainty to human review is safer but loses the Suica-like experience. ZEROGATE is useful only in the middle: preserve the local path when the action, holder, authority, state, and receipt evidence are fresh; otherwise slow down, block, or mark unsupported.

14.13 Operations-research routing

Recognition is still not enough. Once the gate has separated primary actions, context baggage, dependent subactions, tailgating actions, collisions, and ambiguous boundaries, it must allocate scarce slow-lane capacity. This is an operations-research problem: minimize critical-path wait and reviewer backlog subject to safety, receipt, fairness, deadline, and capacity constraints. In

ZEROGATE, the OR layer does not grant authority. It can only route, coalesce, defer, prioritize, or deny work that has already been narrowed by the pass. If a route would widen resource, destination, effect, identity, or policy scope, the route is invalid.

The model below uses the same 144,000-item, 1,200-agent action stream over a one-hour planning horizon. Slow-review capacity is fixed at 180 tasks per minute. This is not a production SLA and not observed customer traffic; it is a deterministic stress model that asks what happens when the system is forced to reason about finite queues rather than independent actions. Linear review cannot finish the hour: it leaves 133,200 items in backlog. FIFO exception review clears the hour but makes urgent resource collisions wait behind lower-urgency ambiguity. A priority queue improves critical conflicts, but may starve ambiguous evidence. The ZEROGATE OR router first classifies the stream, folds context into receipts, contains dependent actions, blocks piggybacked side effects, coalesces related conflicts, and then schedules remaining slow-lane work by deadline and risk.

Table 33: Operations-research routing model over the same 144,000-item action stream. The model tests capacity-aware scheduling, not production traffic.

Policy	Reviewed	Review tasks	Backlog	P95 crit. wait	P95 ambig. wait	Interpretation
Linear full review	144,000	144,000	133,200	760.0	760.0	Every context item, dependency, and side effect enters the same review queue.
FIFO exception review	9,000	9,000	0	31.5	31.5	Only exceptions are reviewed, but urgent collisions still wait behind low-urgency ambiguity.
Priority exception review	9,000	9,000	0	5.7	42.0	Critical conflicts improve, but ambiguous evidence can be starved.
ZEROGATE OR router	9,000	2,000	0	2.4	8.6	Related conflicts are coalesced; deadline and risk scheduling bound both critical and ambiguous waits.

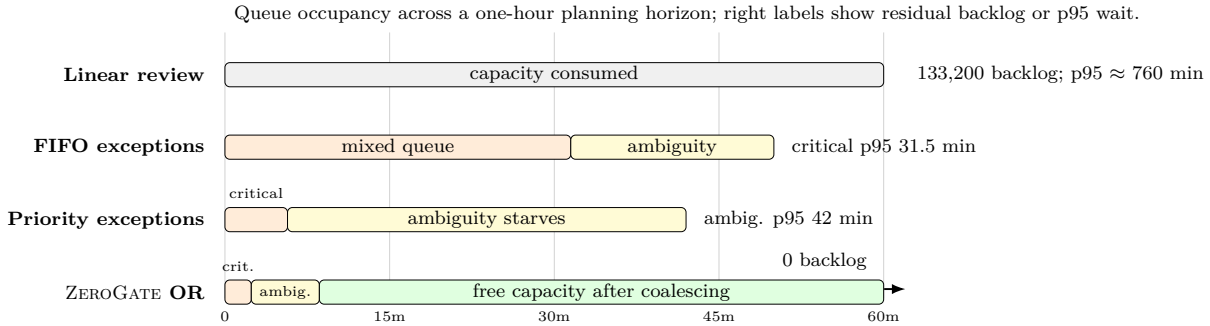


Figure 6: Finite-capacity routing view. A faster local gate alone is not enough at fleet scale; scarce slow-lane capacity must be scheduled. The OR router does not grant authority. It reduces avoidable review load and prioritizes contested work that cannot be locally committed.

This is the operational analogue of not treating every body-shaped movement at a gate as the same event. The system has to know what should move now, what should move with the primary

Table 34: ZEROGATE OR routing decisions. Local routes do not consume scarce slow-review capacity; slow routes are coalesced and scheduled.

Route	Input class	Count	Tasks	Scheduler
Local fast gate	Primary action	72,000	–	Local pass consumption.
Receipt fold	Context baggage	36,000	–	Attach to the primary receipt before or after commit according to evidence profile.
Dependent containment	Dependent subaction	18,000	–	Inherit narrowed authority; cannot widen resource, destination, or effect.
Fail closed	Tailgating action	9,000	–	Terminal block unless an explicit operator appeal is opened.
Slow critical	Crowd collision	6,000	1,000	Earliest-deadline-first with per-resource batching.
Slow batch	Ambiguous boundary	3,000	1,000	Risk-weighted batch review.
Defer until capacity	Evidence enrichment	36,000	–	Background settlement queue after minimum receipt is persisted.

action, what should be stopped, and what should wait for a controlled slot. In the model, 93.75% of items reach a local terminal outcome, slow review tasks are reduced by 77.7778% after coalescing, and reviewer capacity used by the OR router is 18.52% of the one-hour capacity. Compared with linear review, 98.6111% of reviewer capacity is freed for genuinely contested work. The important point is not that these exact numbers transfer to every enterprise. The point is that action governance at fleet scale needs routing and scheduling semantics in addition to action semantics.

14.14 Interpretation

The exact-lane result should be read as a regression result over the specified test oracle, not as a universal safety result. The oracle is simple because the paper is testing pass mechanics rather than semantic parser coverage. The local overhead numbers are JavaScript evaluator timings on the local runner. They exclude adapter parsing, durable storage, remote approval, external verifier calls, queueing, network latency, UI approval time, and proof enrichment.

The important empirical point is architectural. Once a pass is minted, the gate can decide common cases from local material: issuer envelope, hash, fingerprint, holder/session evidence, authority subset, policy digest, state digest, nonce, budget, expiry, risk class, and receipt persistence. Remote proof is moved out of the critical path without losing the binding between the action and the later receipt. The scenario baseline model makes the tradeoff explicit: raw tool permission and audit-only designs can be zero-wait but admit stale or drifted actions under the benchmark oracle; full synchronous governance can preserve the oracle but blocks every action; ZEROGATE preserves the oracle in this benchmark while making common actions locally consumable.

15 Comparison With Existing Layers

Recent vendor systems validate the need for this boundary while also illustrating why an Action Pass is a separate object. Anthropic’s Claude Code auto mode is a clear signal that repeated manual permission prompts create fatigue; its reported design routes tool calls through a classifier that blocks irreversible, destructive, or out-of-environment actions while reducing user interruption [2]. That is a strong harness-level control, but it is still primarily a runtime decision mechanism. ZeroGate asks a different question: after the appropriate runtime or policy layer has decided an action may proceed, what compact object does the final boundary consume, and what receipt proves that the consumed object matched the action that actually ran?

Table 35: Action Passes are not a replacement for runtime protocols or identity layers. They define what crosses the boundary.

Layer	What it answers	What it does not answer alone
OAuth, mTLS, API keys	Who can authenticate to a service?	Whether this exact side effect is still within approved action authority.
A2A	Which agent can communicate or delegate?	Whether the delegated action carries bounded authority and receipt obligations.
MCP	Which tool can be exposed and called?	Whether a concrete call matches the approved action fingerprint and policy state.
Harness	How are tools, memory, retries, and workflow orchestrated?	Whether approval still binds after retries, rewrites, and runtime substitutions.
Audit log	What record was stored afterward?	Whether the record was the gate condition rather than a post-hoc attachment.
Action Pass	Can this action cross this boundary now?	Whether the upstream task is good, useful, or strategically correct.

The practical response to A2A is therefore precise. A2A can say which agent arrived at the gate. An Action Pass says which bounded action ticket the agent is holding, whether it is fresh, whether it matches the action being executed, and whether it can be consumed.

15.1 Interoperability contract

An Action Pass can be carried by a harness, MCP client, A2A message, workflow engine, or tool gateway, but the carrier is not the source of authority. The minimum interoperable payload is: pass identifier, issuer key reference, signed envelope or signature, holder and runtime-session binding, allowed CAVA fingerprint, granted authority tuple, policy digest, state digest, validity interval, nonce, budget, commit profile, evidence profile, and pass hash. The minimum runtime observation returned to OSuite is: observed CAVA fingerprint, observed holder/session evidence, required authority evidence, current policy/state digests, nonce consumption result, receipt sink status, persisted-before-commit status, lane, decision, reason, and receipt hash. If the carrier cannot provide one of these fields, the result is not “best effort fast lane”; it is slow review, fail closed, or observe-only exposure depending on the adapter’s enforcement level.

This contract is complementary to provenance and attestation systems. Proof-carrying code established the older idea that executable artifacts can carry machine-checkable evidence [9]; in-toto and Sigstore focus on supply-chain integrity and signing [4, 16]; W3C Verifiable Credentials define portable credential data models [18]; Dapr’s verifiable-execution work illustrates a newer cloud-native direction for signed execution evidence [3]; and OpenTelemetry-style GenAI conventions describe trace semantics for AI systems [11]. ZeroGate borrows the discipline of verifiable artifacts, but applies it to a narrower runtime question: what bounded action object is consumed before an agent side effect crosses the boundary?

16 Failure Modes and Limits

Action Passes do not solve every governance problem.

- **Bad upstream policy.** A pass can faithfully encode the wrong policy. The gate protects binding and freshness, not policy wisdom.
- **False canonicalization.** If CAVA maps an action incorrectly, the pass inherits that error. This is why parser coverage and abstention must be measured separately.
- **Unlabeled public corpus.** Public workflow artifacts can show coverage, abstention, unsupported surfaces, and routing distribution, but they cannot establish semantic accuracy or false-allow rate without independent labels. The artifact now emits a labeling pack, but the labels and inter-annotator agreement remain future work.
- **Fixture-only multi-runtime result.** The multi-runtime corpus demonstrates boundary-semantics replay across runtime shapes, not empirical parser accuracy on unconstrained runtime traffic.
- **Weak runtime integration.** Observe-only runtimes can produce evidence but may not block before side effect. The system must disclose this.
- **Unsigned or weakly signed passes.** A hash can detect local mutation, but production authorization requires an issuer signature and key-management policy.
- **Holder or session confusion.** A pass copied into another agent, workspace, or runtime session must fail closed; otherwise it becomes a transferable bearer token.
- **Dishonest state digest.** If state digest production is compromised, the pass may bind to bad state. External attestation can reduce this risk but not eliminate all trust.
- **Overbroad pass minting.** If minting policy grants too much scope or time, local checking remains correct but business risk increases.
- **Revocation propagation.** Short validity intervals reduce stale-pass risk, but emergency revocation still requires a revocation channel or slow-lane policy for sensitive action classes.
- **Aggregate-budget splitting.** A single pass can be locally valid while a sequence of passes exceeds a team, tenant, account, wallet, or daily quota unless aggregate budget evidence is checked at commit.
- **State-predicate ambiguity.** A digest can prove that some state snapshot was used, but a reviewer may still need named predicates, such as account status, resource version, balance floor, or branch-protection state, to understand why the action remained safe.
- **Privacy leakage through pass material.** A pass should carry minimized commitments and labels. If raw customer context, secrets, or regulated data are embedded in the pass itself, the proof artifact may become a privacy risk.
- **Human bypass.** Operators can move outside the governed path. AREG should expose missing lanes and uncontrolled surfaces.

- **Compliance interpretation.** The receipt can make a boundary decision reviewable; it does not, by itself, determine legal or regulatory compliance.

These limits are part of the design, not exceptions. A useful governance system should say where it is strong, where it is observe-only, and where it has no evidence.

Claim-boundary table. Where ZEROGATE deliberately refuses to claim completeness.

Open risk	Why it matters	Required control
Pass minting quality	A fast gate faithfully consumes whatever authority was granted. If the issuer grants too much, the gate can be fast and still wrong.	Independent policy review, narrow defaults, short validity, and audit of minted pass distributions.
Issuer authenticity	A <code>pass_hash</code> protects local bytes but not issuer trust.	Signed pass envelopes, key rotation, issuer registry, and verification at commit.
Canonicalization error	A pass binds to a CAVA fingerprint. Incorrect runtime-to-action mapping can bind the wrong semantic object.	Parser coverage metrics, abstention on incomplete evidence, held-out runtime mappings, and external review of action families.
Public-corpus labeling	Real public workflow records are useful, but unlabeled records cannot establish semantic correctness by themselves.	Report coverage, abstention, unsupported cases, and routing distribution separately from future labeled accuracy; use the emitted 240-record labeling pack for independent labels and inter-annotator agreement.
Multi-runtime fixtures	A cross-runtime fixture may be mistaken for real parser coverage.	Keep fixture claims separate from the public corpus; use it only to test whether frozen boundary semantics survive equivalent runtime projections.
Policy/state freshness	A fresh pass can become stale when account, policy, resource, or risk state changes after minting.	Policy and state digests, revocation channels, bounded clock skew, and slow-lane routing on digest drift.
State predicate ambiguity	A state digest may be too opaque for local semantic review when the pass depends on a named condition.	Named predicates, predicate evidence, and slow-lane routing when predicate evidence is missing or drifted.
Revocation freshness	A pass can be valid under time bounds but invalidated by emergency operator or issuer action.	Revocation epochs, revoked-pass/action lists, and slow-lane routing on epoch drift.
Distributed replay	Nonces and budgets are security objects, not annotations. Caller-side replay lists are insufficient in multi-runtime deployments.	Atomic nonce and budget consumption, lease stores, and idempotent receipt sequences.
Aggregate quota	Many individually valid passes can exceed a shared spend, quota, or wallet budget.	Aggregate budget keys, shared budget evidence, and fail-closed behavior when remaining aggregate budget is insufficient.
Holder/session binding	A copied pass must not become transferable authority across agents or sessions.	Runtime-session attestation and adapter-level holder evidence before commit.
Receipt availability	If the receipt path is unavailable, a side effect can become unverifiable even if the action was otherwise allowed.	Local append-only buffers, fail-closed classes, delayed reconciliation, and explicit degraded-mode policy.
Privacy/minimization	Evidence can become a data-leak vector if pass material carries raw sensitive context.	Minimized pass material, hashed or redacted predicates, and slow-lane review when cleartext material appears.
Adapter enforcement level	Some integrations can observe but cannot block before the side effect.	Public adapter-mode disclosure: enforce-before-execute, enforce-with-hook, or observe-only.
Performance generality	A local evaluator benchmark does not imply end-to-end production latency across all customers and tools.	Separate local evaluator metrics from hosted canaries, external SaaS runs, and customer SLA measurements.

The table is intentionally conservative. The pass is not a magic token that makes agents safe. It is a narrow commit object that becomes useful only when minted by a trustworthy authorization layer, bound to a correct action identity, consumed with atomic replay controls, and recorded before side effect.

Validation-hooks table. A serious deployment should be able to reproduce these checks rather than rely on the paper’s wording.

Risk	Falsification test	Current artifact coverage	Evidence still needed
Pass minting quality	Ask the issuer to mint passes for requests that exceed policy, destination, amount, or time limits; measure overbroad-grant rate.	The evaluator blocks runtime-required authority that widens beyond a granted pass.	A minting benchmark over real policy templates and operator-approved scopes.
Issuer authenticity	Alter pass fields or present an unsigned pass; verify that production mode refuses fast-lane consumption.	Reference evaluator detects pass-hash tampering.	Signed envelope verification and key-registry tests.
Canonicalization error	Hold out runtime mappings and check whether equivalent MCP, SDK, shell, browser, and workflow actions produce equivalent fingerprints while boundary violations diverge.	Existing CAVA and skill-boundary artifacts separate mapping accuracy from authorization accuracy; the new multi-runtime fixture reports both metrics separately.	Larger held-out multilingual and framework-specific runtime corpora.
Policy/state freshness	Mint a valid pass, mutate policy or resource state, then attempt fast-lane commit under the stale digest.	Stress cases route policy and state digest drift to slow review.	Live canary tests where policy and SaaS state actually change between mint and commit.
State predicate ambiguity	Require named state predicates in the pass, omit or mutate those predicates at commit, and check whether fast lane is refused.	Stress cases route missing or drifted state predicates to slow review.	Domain-specific predicate libraries and reviewer-authored predicate sets.
Revocation freshness	Mint a valid pass, update the revocation epoch or revoke the pass id, then attempt local commit.	Stress cases slow-lane revocation-epoch drift and block explicit pass revocation.	Live revocation-channel and key-registry propagation tests.
Distributed replay	Submit the same pass/nonce through parallel runtimes and workers; check whether more than one side effect can consume it.	Load study blocks replayed nonces in the evaluator path.	Shared atomic nonce and budget store under multi-process or multi-region deployment.
Aggregate quota	Mint individually valid passes that together exceed a shared budget; check whether the final boundary sees aggregate exhaustion.	Stress cases and load study block aggregate-budget exhaustion.	Shared quota service or ledger-backed budget evidence.
Holder/session binding	Present a valid pass from a different agent, workspace, or runtime session.	Stress cases and load study block holder/session mismatches.	Runtime-session attestation in each enforce-before-execute adapter.
Receipt availability	Disable or degrade the receipt sink and attempt a side effect.	Reference evaluator blocks missing synchronous receipt sinks, blocks missing persisted-before-commit evidence, and reports zero partial commits.	Durable local buffer, reconciliation, and fault-injection runs in hosted Studio.
Privacy/minimization	Add raw customer or secret-like material to pass boundary fields that should be commitments; verify the fast path is refused.	Stress cases and load study route privacy-boundary exposure to slow review.	Redaction, secret scanning, and privacy-safe proof-bundle policy.
Adapter enforcement level	Run the same action through enforce-before-execute and observe-only adapters; verify whether the side effect can occur before OSuite denial.	AREG compatibility text requires exposure disclosure.	Public adapter matrix showing enforce, hook-mediated, and observe-only coverage per runtime.

Risk	Falsification test	Current artifact coverage	Evidence still needed
External side-effect binding	Execute a bounded action against a real third-party SaaS system and compare the resulting object hash with the approved action commitment.	The Baby Blue / invinoveritas v11 canary binds a CAVA artifact to signed ledger entry 246 and verifies that GitHub issue 2’s body hash matches the approved issue-body hash.	Repeated tenant canaries across more SaaS, cloud, wallet, and workflow adapters with independent timestamp and caller-provenance evidence.
Performance generality	Repeat the benchmark under hosted, networked, and external SaaS side-effect paths; compare local evaluator latency against end-to-end operator wait.	Local batch and concurrent load studies exercise the real evaluator, plus a hosted Studio action-recording smoke.	Tenant canaries, external SaaS mutation runs, and SLA-style latency distributions.

These validation hooks separate three claims that are easy to conflate. First, the local evaluator can preserve lane decisions over the specified pass semantics. Second, OSuite’s hosted action-recording surface can persist runtime evidence. Third, a deployed customer topology can enforce the same boundary before a real side effect. This paper provides evidence for the first claim and a narrow smoke check for the second. The third requires deployment-specific adapter and storage evidence.

16.1 Claim-to-Evidence Summary

The paper is written to make the main claims falsifiable rather than merely persuasive. The following table maps each central claim to the evidence available in the artifact and to the remaining boundary of interpretation. The table is not a substitute for peer review; it is a compact index of what the current evidence does and does not establish.

Claim-to-evidence table. Claim-to-evidence map for the current artifact.

Claim area	Potential ambiguity	Evidence boundary
Novelty	Difference from a permission check or audit log.	The object of control is a bounded action pass tied to CAVA fingerprint, authority tuple, holder/session, freshness, revocation, budget, privacy posture, and receipt condition.
Technical mechanism	Whether the contribution is only architecture prose.	The paper defines pass and observation tuples, lane function, authority coverage, narrowing, replay, freshness, receipt, privacy, and aggregate-budget properties, then implements the evaluator.
Baseline validity	Whether modeled baselines overstate competing weaknesses.	The paper separates modeled design-pattern comparisons from a measured local decision-function comparison. On the same 360 inputs, raw tool-permission and broad session checks are faster but false-allow 0.9000 and 0.8333 of non-fast oracle cases respectively; the ZeroGate evaluator false-allows 0.
Evaluation scale	Whether the benchmark is too small or handpicked.	The artifact separates authored stress-oracle cases from other validation views: 5,760 weighted synthetic observations, 360 adversarial stress cases, 528 multi-runtime fixture cases, 2,500 real public workflow records, a 240-record pending labeling pack, a 100 ms passage-envelope model, 700,000 load-study evaluations, a 144,000-item action-crowd model, a 72,750-record noisy-evidence calibration, and a one-hour OR routing model.
Dataset auditability	Whether the reported dataset can be recomputed.	The artifact includes benchmark-spec, stress-manifest, representative-manifest, multi-runtime case-manifest, source-snapshot, derived-record, and labeling-pack checksums; frozen family/mutation splits; repository-level public-corpus splits; Wilson intervals; and an oracle-leakage audit confirming expected labels and benchmark mutation metadata are not present in evaluator inputs.

Claim area	Potential ambiguity	Evidence boundary
Production-path evidence	Whether all evidence is synthetic or local-only.	The paper separates public workflow artifacts, a hosted Studio action-recording smoke check, and a public Baby Blue / invinoveritas v11 plus GitHub issue side-effect canary. These support live-path reachability and proof binding, not customer traffic rates.
Independent labeling	Whether public-corpus coverage is being misreported as semantic accuracy.	The paper freezes a 240-record labeling pack and states the annotator, blinding, agreement, adjudication, and freeze protocol before any labeled accuracy can be claimed. Current public-corpus results remain coverage and abstention results only.
End-to-end SLA	Whether local microbenchmarks imply customer latency.	The paper explicitly separates local commit latency from prepare, receipt, external tool execution, verifier, proof enrichment, queueing, and hosted canary clocks; <code>production_sla_claim=false</code> remains part of the artifact.
Recognition and scheduling	Whether higher throughput hides boundary mistakes.	Action crowd control measures whether context baggage, dependent subactions, tailgating actions, collisions, and ambiguous boundaries are separated; noisy calibration measures conservative degradation under incomplete evidence; OR routing measures scarce slow-lane capacity allocation without widening authority.
Passage-envelope control	Whether the railway analogy overclaims performance.	The 100 ms envelope is a stated design budget; measured values cover only the warmup batch local evaluator. Curvature profiles show that common and moderate traffic fit after active support, while policy churn and emergency revocation intentionally miss the envelope because safety requires slow review or stop.
Reproducibility	Whether a reader can rerun the evidence.	The benchmark, public-runtime corpus snapshot, labeling pack, multi-runtime fixture, load study, schema, source notes, and commands are included in the repository; timing is labeled diagnostic rather than production SLA.
External validity	Scope of deployment-level safety claims.	The paper limits claims to the reference evaluator and states required production evidence for issuer signatures, atomic stores, adapter enforcement, hosted canaries, and external SaaS runs.
Relation to OSuite primitives	Compatibility with CAVA, PCAA, BAF, AREG, and skill-boundary research.	ZeroGate consumes their outputs at the runtime boundary; vendor-neutral skill boundaries request authority, PCAA grants it, CAVA fingerprints it, and ZeroGate consumes the narrowed pass.

16.2 Production hardening requirements

The reference evaluator is intentionally small so that the pass semantics are inspectable. A production issuer must add deployment controls around it rather than confuse the reference hash with a complete trust boundary. In particular, production Action Passes should be signed using a verifier-friendly envelope such as JWS or COSE, issuer keys should rotate with explicit key identifiers, revocation epochs should be explicit, nonce and budget consumption should use an atomic store, aggregate budgets should be checked against shared quota evidence, receipt sinks should provide append-only or tamper-evident write semantics, clocks should tolerate bounded skew, sensitive pass material should be minimized or committed by hash, and runtime adapters should disclose whether they are enforce-before-execute or observe-only. These requirements do not change the ZEROGATE semantics. They are the operational machinery needed to make the semantics reliable under real deployment

conditions.

16.3 Boundary with vendor-native control planes

Large platform vendors can implement parts of this design inside their own harnesses, agent managers, or control planes. That does not make the idea invalid; it changes the trust question. A vendor-native control plane can be a strong enforcement point for that vendor’s own agents and tools, but it may not be the independent verifier or cross-runtime proof layer an enterprise wants when actions span OpenAI, Claude, Microsoft, MCP servers, SaaS workflows, shell commands, and internal systems. ZeroGate is therefore not positioned as a replacement for native controls. It is the portable action-boundary object that can sit across them: the acting party may be one system, the verifier another, the runtime a third, and the later auditor a fourth.

17 Research Roadmap

The immediate next step is to enlarge the benchmark while keeping the core semantics frozen. The expansion should include:

1. more action families across SaaS mutation, cloud identity, data export, code deployment, payment, wallet, and workflow lanes;
2. held-out runtime mappings so the schema is not tuned only to the examples used during design;
3. separate measurement of runtime-to-action mapping accuracy and authorization-decision accuracy;
4. repeated end-to-end canary runs with real side effects, public proof bundles, independently timestamped verifier entries, and adapter-specific SLA distributions;
5. comparison against raw tool permission, session approval, IAM-only gating, audit-only controls, and harness-native approvals;
6. formal treatment of scope narrowing for operation, effect, resource, destination, and constraint tuples.

The most important theoretical question is not whether a pass can be checked locally. That is straightforward. The harder question is how to define the partial order over authority scopes so that runtime evidence may narrow authority but never widen it. This is also where vendor-neutral skill-boundary work can connect naturally: a skill may request authority, but the runtime must decide the actual granted pass.

A second research question is operational rather than semantic: how much of the pass lifecycle can be made independently recomputable? In the strongest version, a third party should be able to inspect the requested skill boundary, the granted pass, the CAVA fingerprint, policy and state digest references, nonce consumption record, receipt hash, and final proof bundle without trusting the acting agent’s transcript. This is where ZEROGATE connects back to proof-carrying agent actions rather than becoming only a performance optimization.

18 Conclusion

AI-agent governance needs a faster common path and a stricter exception path. Remote approval for every action is too heavy; broad trust in tools, sessions, or agent identity is too weak. ZEROGATE Action Passes propose a middle object: compiled authority for a specific canonical action, locally consumable at the runtime boundary, bounded by policy and state digests, protected against replay, and tied to a receipt that can support later proof.

The design lesson from FeliCa/Suica-style gates is not merely speed. It is discipline: common passage should be local, deterministic, and bounded; exceptions should be explicit; settlement should not erase what the gate decided. For AI agents, that translates into a simple rule. Agents should not ask for permission from scratch every time. They should carry bounded action passes.

A Reference Pseudocode

```
evaluate(pass, observation, now):
    if production_mode and !verify_issuer_envelope(pass):
        return block("issuer_envelope_invalid")
    if pass.hash != hash(pass without pass_hash):
        return block("invalid_action_pass")
    if now < pass.not_before or now > pass.expires_at:
        return block("pass_not_current")
    if observation.holder != pass.holder:
        return block("holder_identity_mismatch")
    if pass.runtime_session and
        observation.runtime_session != pass.runtime_session:
        return block("runtime_session_mismatch")
    if pass.id in revoked_passes or
        pass.allowed_fingerprint in revoked_actions:
        return block("pass_revoked")
    if pass.nonce in consumed_nonces:
        return block("replay_nonce_seen")
    if observation.fingerprint != pass.allowed_fingerprint:
        return block("approval_execution_divergence")
    if observation.requested_units > pass.budget.remaining:
        return block("budget_exhausted")
    if observation.requested_units >
        observation.aggregate_budget_remaining:
        return block("aggregate_budget_exhausted")
    if observation.required_authority is missing:
        return slow("authority_evidence_missing")
    if !subset(observation.required_authority,
        pass.authority.granted):
        return block("authority_scope_widened")
    if observation.policy_digest != pass.policy_digest:
        return slow("policy_digest_drift")
    if observation.state_digest != pass.state_digest:
        return slow("state_digest_drift")
```

```

if pass.revocation_epoch and
    observation.revocation_epoch != pass.revocation_epoch:
    return slow("revocation_epoch_drift")
if required_state_predicate_missing(observation):
    return slow("state_predicate_evidence_missing")
if required_state_predicate_drifted(observation):
    return slow("state_predicate_drift")
if pass.requires_minimized_material and
    observation.pass_material_exposed:
    return slow("privacy_boundary_review_required")
if pass.requires_sync_receipt and !receipt_sink_available:
    return block("receipt_sink_unavailable")
if pass.requires_sync_receipt and !receipt_persisted_before_commit:
    return block("receipt_not_persisted_before_commit")
if observation.time_slot_conflict:
    return slow("time_slot_collision_detected")
if pass.remote_verifier_required:
    return slow("remote_verifier_required")
if pass.commit_lane != "fast":
    return slow("not_compiled_for_fast_lane")
if pass.risk_class in ["high", "critical"]:
    return slow("risk_exceeds_fast_lane")
return fast("local_commit_satisfied")

```

B Example Pass Skeleton

```

{
  "schema_version": "osuite.action_pass.v1",
  "pass_id": "ap...",
  "issuer": {"id": "osuite-runtime-authorizer"},
  "holder": {
    "agent_id": "agent:ops-bot",
    "principal_id": "operator:jw-ond",
    "workspace_id": "workspace:osuite",
    "runtime_session_id": "session:agent-run-123"
  },
  "authority": {
    "narrowing_rule": "runtime_may_narrow_never_widen",
    "granted": {
      "operations": ["issue_refund"],
      "resource_scope": ["order:ORD-73"],
      "destinations": ["payment_ledger"],
      "constraints": {"max_amount_usd": 100}
    }
  },
  "action_binding": {
    "mode": "cava_fingerprint",

```

```

    "allowed_fingerprint": "...",
  },
  "policy_binding": {"policy_digest": "..."},
  "state_binding": {"state_digest": "..."},
  "budget": {"unit": "action", "limit": 1, "consumed": 0},
  "validity": {"not_before": "...", "expires_at": "..."},
  "nonce": "...",
  "commit_profile": {"lane": "fast"},
  "evidence_profile": {
    "synchronous_receipt": true,
    "receipt_sink": "local_append_only_buffer"
  },
  "pass_hash": "...",
  "signature": {
    "format": "deployment-envelope",
    "key_ref": "osuite:key:prod",
    "value": "..."
  }
}

```

References

- [1] A2A Project. Agent2agent protocol. <https://github.com/a2aproject/A2A>, 2026.
- [2] Anthropic. Auto mode is now the default in claude code for pro, max, and team plans. <https://claude.com/blog/auto-mode-default-in-claude-code>, 2026.
- [3] Cloud Native Computing Foundation. Introducing verifiable execution in dapr 1.18. <https://www.cncf.io/blog/2026/06/11/introducing-verifiable-execution-in-dapr-1-18/>, 2026.
- [4] in-toto project. in-toto: A framework for securing the integrity of software supply chains. <https://in-toto.io/>, 2023.
- [5] JR East Information Systems Company. Suica and station services solutions. <https://www.jerail.co.jp/en/solution/suica/>, n.d. Accessed 2026-09-20.
- [6] Microsoft. Microsoft agent 365: The control plane for ai agents. <https://www.microsoft.com/en-us/copilot/blog/2025/11/18/microsoft-agent-365-the-control-plane-for-ai-agents/>, 2025.
- [7] Model Context Protocol contributors. Model context protocol authorization. <https://modelcontextprotocol.io/specification/draft/basic/authorization>, 2026.
- [8] National Institute of Standards and Technology. Announcing the ai agent standards initiative for interoperable and secure innovation. <https://www.nist.gov/news-events/news/2026/02/announcing-ai-agent-standards-initiative-interoperable-and-secure>, 2026.
- [9] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119, 1997.

- [10] OpenAI. Openai agents sdk. <https://openai.github.io/openai-agents-python/>, 2026.
- [11] OpenTelemetry. Semantic conventions for generative ai. <https://github.com/open-telemetry/semantic-conventions-genai>, 2026.
- [12] OWASP GenAI Security Project. Owasp top 10 for agentic applications for 2026. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>, 2026.
- [13] Hiroyuki Sakai. Air spring tilting control system for rolling stock. *Journal of The Society of Instrument and Control Engineers*, 56(2):123–126, 2017. doi: 10.11499/sicejl.56.123.
- [14] Akio Shiibashi. Development of jr east’s suica non-contact ic card. *Japan Railway & Transport Review*, 32:20–25, 2002.
- [15] Akio Shiibashi. Development of suica autonomous decentralized ic card ticket system. *Japan Railway & Transport Review*, 50:6–15, 2008.
- [16] Sigstore. Sigstore documentation. <https://docs.sigstore.dev/>, 2024.
- [17] Sony Corporation. Felica contactless ic card system. <https://www.sony.co.jp/en/Products/felica/about/scheme.html>, n.d. Accessed 2026-09-20.
- [18] W3C. Verifiable credentials data model v2.0. <https://www.w3.org/TR/vc-data-model-2.0/>, 2025.
- [19] Zexun Wang. Cava: Canonical action verification and attestation for runtime governance of agentic ai systems. <https://arxiv.org/abs/2607.13040>, 2026.
- [20] Zexun Wang. Proof-carrying agent actions: Model-agnostic runtime governance for heterogeneous agent systems. <https://arxiv.org/abs/2606.04104>, 2026.
- [21] Katsumi Yagishita. JR-East shinkansen technology. https://www.schienenfahrzeugtagung.at/download/PDF2013/02_Yagishita.pdf, 2013. Accessed 2026-09-21.